

# Explaining Deep Neural Networks and Beyond: A Review of Methods and Applications

*This review provides a timely overview of explainable AI for deep neural networks, with a focus on post hoc explanations.*

By WOJCIECH SAMEK<sup>ID</sup>, Member IEEE, GRÉGOIRE MONTAVON<sup>ID</sup>, SEBASTIAN LAPUSCHKIN<sup>ID</sup>, CHRISTOPHER J. ANDERS, AND KLAUS-ROBERT MÜLLER<sup>ID</sup>, Member IEEE

**ABSTRACT** | With the broader and highly successful usage of machine learning (ML) in industry and the sciences, there has been a growing demand for explainable artificial intelligence (XAI). Interpretability and explanation methods for gaining a better understanding of the problem-solving abilities and strategies of nonlinear ML, in particular, deep neural networks, are, therefore, receiving increased attention. In this work, we aim to: 1) provide a timely overview of this active

emerging field, with a focus on “post hoc” explanations, and explain its theoretical foundations; 2) put interpretability algorithms to a test both from a theory and comparative evaluation perspective using extensive simulations; 3) outline best practice aspects, i.e., how to best include interpretation methods into the standard usage of ML; and 4) demonstrate successful usage of XAI in a representative selection of application scenarios. Finally, we discuss challenges and possible future directions of this exciting foundational field of ML.

**KEYWORDS** | Black-box models; deep learning; explainable artificial intelligence (XAI); Interpretability; model transparency; neural networks.

Manuscript received March 4, 2020; revised December 21, 2020; accepted February 7, 2021. Date of current version March 3, 2021. This work was supported in part by the Institute of Information & Communications Technology Planning & Evaluation (IITP) grants funded by the Korea Government under Grant 2017-0-00451 (Development of BCI Based Brain and Cognitive Computing Technology for Recognizing User’s Intentions using Deep Learning) and Grant 2019-0-00079 (Artificial Intelligence Graduate School Program, Korea University); in part by the German Ministry for Education and Research (BMBF) under Grant 01IS14013A-E, Grant 01GQ1115, Grant 01GQ0850, Grant 01IS18025A, and Grant 01IS18037A; and in part by the German Research Foundation (DFG) under Grant Math+, EXC 2046/1 and Project 390685689. (Wojciech Samek and Grégoire Montavon contributed equally to this work.) (Corresponding authors: Wojciech Samek; Grégoire Montavon; Klaus-Robert Müller.)

**Wojciech Samek** is with the Department of Artificial Intelligence, Fraunhofer Heinrich Hertz Institute, 10587 Berlin, Germany, and also with the BIFOLD—Berlin Institute for the Foundations of Learning and Data, 10587 Berlin, Germany (e-mail: wojciech.samek@hhi.fraunhofer.de).

**Grégoire Montavon** and **Christopher J. Anders** are with the Machine Learning Group, Technische Universität Berlin, 10587 Berlin, Germany, and also with the BIFOLD—Berlin Institute for the Foundations of Learning and Data, 10587 Berlin, Germany (e-mail: gregoire.montavon@tu-berlin.de).

**Sebastian Lapuschkin** is with the Department of Artificial Intelligence, Fraunhofer Heinrich Hertz Institute, 10587 Berlin, Germany.

**Klaus-Robert Müller** is with the Machine Learning Group, Technische Universität Berlin, 10587 Berlin, Germany, with the Department of Artificial Intelligence, Korea University, Seoul 136-713, South Korea, with the Max Planck Institute for Informatics, 66123 Saarbrücken, Germany, and also with the BIFOLD—Berlin Institute for the Foundations of Learning and Data, 10587 Berlin, Germany (e-mail: klaus-robert.mueller@tu-berlin.de).

Digital Object Identifier 10.1109/JPROC.2021.3060483

## I. INTRODUCTION

The main goal of machine learning (ML) is to learn accurate decision systems, respectively, predictors that can help automatizing tasks, which would otherwise have to be done by humans. ML has supplied a wealth of algorithms that have demonstrated important successes in the sciences and industry; most popular ML workhorses are considered to be kernel methods (e.g., [132], [163], [164], [190], and [194]), and particularly during the last decade, deep learning methods (e.g., [23], [52], [70], [107], [108], and [161]) have gained highest popularity.

As ML is increasingly used in real-world applications, a general consensus has emerged that high prediction accuracy alone may not be sufficient in practice [28], [104], [159]. Instead, in practical engineering of systems, critical features that are typically considered beyond excellent prediction itself are: 1) robustness of the system to

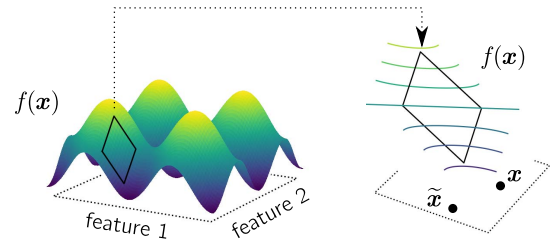
measurement artifacts or adversarial perturbations [182]; 2) resilience to drifting data distributions [49]; 3) ability to accurately assess the confidence of its own predictions [135], [139]; 4) safety and security aspects [21], [26], [84], [193]; 5) legal requirements or adherence to social norms [54], [60]; 6) ability to complement human expertise in decision-making [82]; or 7) ability to reveal to the user the interesting correlations that it has found in the data [88], [165].

Orthogonal to the quest for better and more holistic ML models, explainable artificial intelligence (XAI) [18], [73], [112], [128], [159] has developed as a subfield of ML that seeks to augment the training process, the learned representations, and the decisions with human-interpretable explanations. An example is *medical diagnosis*, where the input examples (e.g., histopathological images) come with various artifacts (e.g., stemming from image quality or sub-optimal annotations) that have, in principle, nothing to do with the diagnostic task, yet, due to the limited amount of available data, the ML model may harvest specifically these spurious correlations with the prediction target (e.g., [61] and [177]). Here, interpretability could point at anomalous or awkward decision strategies before harm is caused in a later usage as a diagnostic tool.

Similarly essential when using ML in the sciences is again interpretability since, ideally, the transparent ML model—having learned from data—may have embodied scientific knowledge that would subsequently provide insight to the scientist; occasionally, this can even be novel scientific insight (see [165]). Note that, in numerous scientific applications, it has been most common so far to use linear models [151], favoring interpretability often at the expense of predictivity (see [63] and [117]).

To summarize, there is a strong push toward better understanding ML systems that are being used, and in consequence, blackbox algorithms are more and more abandoned for many applications. This growing consensus has led to strong growth of a subfield of ML, namely, XAI that strives to produce transparent nonlinear learning methods, and supplies novel theoretical perspectives on ML models, along with powerful practical tools for a better understanding and interpretation of AI systems.

In this review article, we will summarize the recent exciting developments, present different classes of XAI methods that have been proposed in the context of deep neural networks, provide theoretical insights, and highlight the current best practices when applying these methods. Note, finally, that we do not attempt an encyclopedic treatment of all available XAI literature; rather, we present a slightly biased point of view (and, in doing so, we often draw from the work of the authors). In particular, we focus on *post hoc explanation methods* that take any model, typically the best performing one, and analyze it in the second step in order to uncover its decision strategy. We also provide—to the best of our knowledge—reference to other related works for further reading.



**Fig. 1.** Example of a nonlinear function of the input features, which produces some prediction. The function can be approximated locally as a linear model.

## II. TOWARD EXPLAINING DEEP NEURAL NETWORKS

Before discussing aspects of the problem of explanation that are specific to deep neural networks, we first introduce some basics of XAI, which apply to a fairly general class of ML models. The ML model will be assumed to be *already trained*, and the input–output relation that it implements will be abstracted by some *function*

$$f: \mathbb{R}^d \rightarrow \mathbb{R}.$$

This function receives as input a vector of real-valued features  $x = (x_1, \dots, x_d)$  typically corresponding to various sensor measurements. The function produces as an output a real-valued score on which the decision is based. A sketch of such function receiving two features  $x_1$  and  $x_2$  as input is given in Fig. 1.

In the context of ML classification, the function output can be interpreted as the amount of evidence for/against deciding in favor of a certain class. A classification decision is then obtained from the output score by testing whether the latter is above a certain threshold or, for multiclass problems, larger than the output score of other functions representing the remaining classes.

In a medical scenario, the function may receive as input an array of clinical variables, and the output of the function may represent the evidence for a certain medical condition [96]. In an engineering setting, the input could be the composition of some compound material, and the output could be a prediction of its strength [197] or stability.

Suppose that a given instance is predicted by the ML model to be healthy, or a compound material is predicted to have high strength. We may choose to trust the prediction and go ahead with the next step within an application scenario. However, we may benefit from taking a closer look at that prediction, e.g., to verify that the prediction “healthy” is associated with relevant clinical information, and not some spurious features that accidentally correlate with the predicted quantity in the data set [101], [104]. Such a problem can often be identified by building an *explanation* of the ML prediction [104].

Conversely, suppose that another instance is predicted by the ML model to be of low health or low strength. Here, an explanation could prove equally useful as it could hint at actions to be taken on the sample to improve its predicted score [189], e.g., possible therapies in a medical setting or small adjustments of the compound design that leads to higher strength.

### A. How to Explain: Global Versus Local

Numerous approaches have emerged to shed light onto ML predictions. Certain approaches, such as activation-maximization [134], [136], [174], aim at a *global* interpretation of the model, by identifying prototypical cases for the output quantity

$$x^* = \arg \max_x f(x)$$

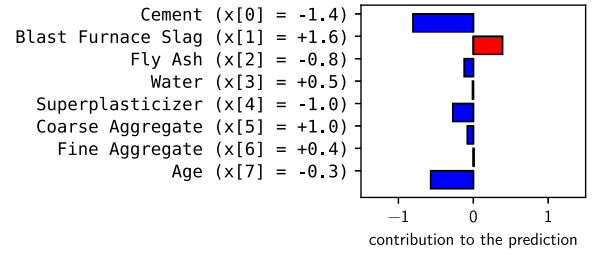
and allowing, in principle, to verify that the function has a high value only for the valid cases. While these prototypical cases may be interesting per se, both for model validation or knowledge discovery, such prototypes will be of little use to understand for a given example  $x$  (say, near the decision boundary) what features play in favor or against the model output  $f(x)$ .

Specifically, we would like to know for that very example what input features contribute positively or negatively to the given prediction. These *local* analyses of the decision function have received growing attention, and many approaches have been proposed [13], [14], [147], [183], [201]. For simple models with limited nonlinearity, the decision function can be approximated locally as the linear function [13]

$$f(x) \approx \sum_{i=1}^d \underbrace{[\nabla f(\tilde{x})]_i \cdot (x_i - \tilde{x}_i)}_{R_i} \quad (1)$$

where  $\tilde{x}$  is some nearby root point (see Fig. 1). This expansion takes the form of a weighted sum over the input features, where the summand  $R_i$  can be interpreted as the contribution of feature  $i$  to the prediction. Specifically, an inspection of the summands reveals that a feature  $x_i$  will be attributed strong relevance if the following two conditions are met: 1) the feature must be expressed in the data, i.e., it differs from the reference value  $\tilde{x}_i$  and 2) the model output should be sensitive to the presence of that feature, i.e.,  $[\nabla f(\tilde{x})]_i \neq 0$ . An explanation for the prediction can then be formed by the vector of relevance scores  $(R_i)_i$ . It can be given to the user as a histogram over the input features or as a heatmap.

For illustration, consider the problem of explaining a prediction for a data point from the Concrete Compressive Strength data set [197]. For this data point, a simple two-layer neural network model predicts a low compressive strength. Applying the analysis above gives an explanation for this prediction, which we show in Fig. 2.



**Fig. 2.** Input example predicted to have low compressive strength and a featurewise explanation of the prediction. Red and blue indicate positive and negative contributions.

For this example, low cement concentration and below average age are factors of low compressive strength although this is partly compensated by a high quantity of blast furnace slag.

Furthermore, for an explanation to be interpretable by its receiver, the latter must be able to make sense of the input features. Some features, such as “cement,” “water,” and “age,” are understandable to everyone; however, more technical terms, such as “blast furnace slag” or “superplasticizer,” may only be accessible to a domain expert. Therefore, when using these explanation techniques, we make the implicit assumption that those input features are interpretable to the receiver.

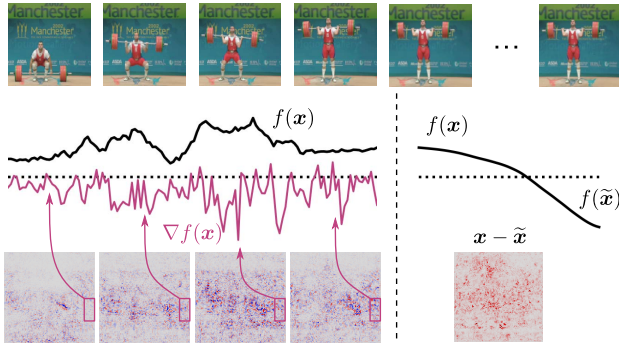
### B. Deep Networks and the Difficulty of Explaining Them

In practice, linear models or shallow neural networks may not be sufficiently expressive to predict the task optimally. Deep neural networks (DNNs) have been proposed as a way of producing more predictive models. They can be abstracted as a sequence of layers

$$f(x) = f_L \circ \dots \circ f_1(x)$$

where each layer applies a linear transformation followed by an elementwise nonlinearity. Combining a large number of these layers endows the model with high prediction power. DNNs have proven especially successful on computer vision tasks [65], [99], [175]. However, DNN models are also much more complex and nonlinear, and quantities entering into the simple explanation model of (1) become considerably harder to compute and estimate reliably.

The first difficulty comes from the multiscale and distributed nature of neural network representations. Some neurons are activated for only a few data points, whereas others apply more globally. The prediction is, thus, a sum of local and global effects, which makes it difficult (or impossible) to find a root point  $\tilde{x}$  that linearly expands to the prediction for the data point of interest. The transition from the global to local effect, indeed, introduces a nonlinearity, which (1) cannot capture.



**Fig. 3.** Two difficulties encountered when explaining DNNs. Left: shattered gradient effect causing gradients to be highly varying and too noisy to be used for explanation. Right: pathological minima in the function, making it difficult to search for meaningful reference points.

The second source of instability arises from the high depth of recent neural networks, where a “shattered gradient” effect was observed [16], noting that the gradient locally resembles white noise. In particular, it can be shown that, for deep rectifier networks, the number of discontinuities of the gradient can grow in the worst case exponentially with depth [129]. The shattered gradient effect is illustrated in Fig. 3 (left) for the well-established VGG-16 network [175]: The network is fed multiple consecutive video frames of an athlete lifting a barbell, and we observe the prediction for the output neuron “barbell.” The gradient of the prediction is changing its value much more quickly than the prediction itself. An explanation based on such gradient would, therefore, inherit this noise.

The last difficulty comes from the challenge of searching for a root point  $\tilde{x}$  on which to base the explanation, which is both close to the data and not an adversarial example [53], [135]. The problem is illustrated in Fig. 3 (right), where we showcase a reference point  $\tilde{x}$  that does not carry any meaningful visual difference to the original data  $x$ , but for which the function output has changed dramatically. The problem of adversarial examples can be explained by the gradient noise that causes the model to “overreact” to certain pixelwise perturbations and also by the high dimensionality of the data ( $224 \times 224 = 50\,176$  pixels for VGG-16 and the ImageNet data set) where many small pixelwise effects cumulate into a large effect on the model output.

### III. PRACTICAL METHODS FOR EXPLAINING DNNs

In view of the multiple challenges posed by analyzing deep neural network functions, building robust and practical methods to explain their decisions has developed into an own research area [59], [128], [159], and an abundance of methods has been proposed. In parallel, efficient software (see Appendix C for a list) makes these newly developed methods readily usable in practice and allows

researchers to perform systematic comparisons between them on reference models and data sets.

In this section, we focus on four families of explanation techniques: interpretable local surrogates, occlusion analysis, gradient-based techniques, and layerwise relevance propagation (LRP). In our view, these techniques exemplify the current diversity of possible approaches to explaining predictions in terms of input features, and, taken together, provide broad coverage of the types of models to explain and the practical use cases. We give references to further related methods in the corresponding subsections. Table 3 provides a glossary of all referenced methods.

#### A. Interpretable Local Surrogates [147]

This category of methods aims to replace the decision function with a local surrogate model that is structured in a way that it is self-explanatory (an example of a self-explanatory model is the linear model). This approach is embodied in the LIME algorithm [147], which was successfully applied to DNN classifiers for images and text. The explanation can be achieved by first defining some local distribution  $p_x(\xi)$  around our data point  $x$ , learning the parameter  $v$  of the linear model that best matches the function locally

$$\min_v \int [f(\xi) - v^\top \xi]^2 \cdot dp_x(\xi)$$

and then extracting local feature contributions, e.g.,  $R_i = v_i x_i$ . Because the method does not rely on the gradient of the original DNN model, it avoids some of the difficulties discussed in Section II-B. The LIME method also covers the incorporation of sparsity or simple decision trees to the surrogate model to further enhance interpretability. In addition, the learned surrogate model may be based on its own set of *interpretable features*, allowing to produce explanations in terms of features that are maximally interpretable to the human. Other methods that explain by building a local surrogate include LORE [58] and Anchors [148]. Furthermore, a broader set of methods does not consider a specific location in the input space and instead builds a global surrogate model of the decision function, where the surrogate model readily incorporates interpretability structures. We discuss these global “self-explainable” models in Section III-E.

#### B. Occlusion Analysis [201]

The occlusion analysis is a particular type of perturbation analysis where we repeatedly test the effect on the neural network output of occluding patches or individual features in the input image [201], [208]

$$R_i = f(x) - f(x \odot (1 - m_i))$$

where  $m_i$  is an indicator vector for the patch or feature to remove and “ $\odot$ ” denotes the elementwise product.



A heatmap  $(R_i)_i$  can be built from these scores highlighting locations where the occlusion has caused the strongest decrease of the function. Because occlusion may produce visual artifacts, inpainting occluded patterns (e.g., using a generative model [2]) rather than setting them to gray was proposed as an enhancement.

Attribution based on Shapley values [116], [179] (see Section V-A for a definition) can also be seen as an occlusion analysis. Here, instead of occluding features one at a time, a much broader set of occlusion patterns is considered, and this has the effect of also integrating global effects in the explanation. SHAP and Kernel SHAP [116] are practical algorithms to approximate Shapley values that sample a few occlusions according to the probability distribution used to compute Shapley values and then fit a linear surrogate model that correctly predicts the effect of these occlusions on the output. An explanation can then be easily extracted, and this explanation retains some similarity with the original Shapley values.

A further extension of occlusion analysis is meaningful perturbation [47], where an occluding pattern is synthesized, subject to a sparsity constraint, in order to engender the maximum drop of the function value  $f$ . The explanation is then readily given by the synthesized pattern. The perturbation-based approach was later embedded in a rate distortion theoretical framework [118].

### C. Integrated Gradients/ SmoothGrad [176], [183]

*Integrated gradients* [183] explain by integrating the gradient  $\nabla f(\mathbf{x})$  along some trajectory in input space connecting some root point  $\tilde{\mathbf{x}}$  to the data point  $\mathbf{x}$ . The integration process addresses the problem of the locality of the gradient information (see Section II-B), making it well-suited for explaining functions that have multiple scales. In the simplest form, the trajectory is chosen to be the segment  $[\tilde{\mathbf{x}}, \mathbf{x}]$  connecting some root point to the data. Integrated gradients define featurewise scores as

$$R_i(\mathbf{x}) = (x_i - \tilde{x}_i) \cdot \int_0^1 [\nabla f(\tilde{\mathbf{x}} + t \cdot (\mathbf{x} - \tilde{\mathbf{x}}))]_i dt.$$

It can be shown that these scores satisfy  $\sum_i R_i(\mathbf{x}) = f(\mathbf{x})$  and, thus, constitute a *complete* explanation. If necessary, the method can be easily extended to any trajectories in input space. For implementation purposes, integrated gradients must be discretized. Specifically, the continuous trajectory is approximated by a sequence of data points  $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(N)}$ . Integrated gradients are then implemented, as shown in Algorithm 1.

The gradient can easily be computed using automatic differentiation. The larger the number of discretization steps, the closer the output gets to the integral form, but the more computationally expensive the procedure gets.

---

#### Algorithm 1 Integrated Gradients

---

```

 $R = 0$ 
for  $n = 1$  to  $N - 1$  do
   $R = R + \nabla f(\mathbf{x}^{(n)}) \odot (\mathbf{x}^{(n+1)} - \mathbf{x}^{(n)})$ 
end for
return  $R$ 

```

---

Another popular gradient-based explanation method is *SmoothGrad* [176]. The function's gradient is averaged over a large number of locations corresponding to small random perturbations of the original data point  $\mathbf{x}$

$$\nabla_{\text{smooth}} f(\mathbf{x}) = \mathbb{E}_{\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, \sigma^2 \mathbf{I})} [\nabla f(\mathbf{x} + \boldsymbol{\varepsilon})].$$

Like the method's name suggests, the averaging process "smooths" the explanation and, in turn, also addresses the shattered gradient problem described in Section II-B (see also [14], [130], and [174] for earlier gradient-based explanation techniques).

In Section IV, we experiment with a combination of integrated gradients and SmoothGrad [176], similar to expected gradients (see [181]), where relevance scores obtained from integrated gradients are averaged over several integration paths that are drawn from some random distribution. The resulting method preserves the advantages of integrated gradients and further reduces the gradient noise.

### D. Layerwise Relevance Propagation (LRP) [13]

The LRP method [13] makes explicit use of the layered structure of the neural network and operates in an iterative manner to produce the explanation. Consider the neural network

$$f(\mathbf{x}) = f_L \circ \dots \circ f_1(\mathbf{x}).$$

First, activations at each layer of the neural network are computed until we reach the output layer. The activation score in the output layer forms the prediction. Then, a reverse propagation pass is applied, where the output score is progressively redistributed, layer after layer, until the input variables are reached. The redistribution process follows a conservation principle analogous to Kirchhoff's laws in electrical circuits. Specifically, all "relevance" that flows into a neuron at a given layer flows out toward the neurons of the layer below. At a high level, the LRP procedure can be implemented as a forward-backward loop, as shown in Algorithm 2.

The function `relprop` performs redistribution from one layer to the layer below and is based on "propagation rules" defining the exact redistribution policy. Examples of propagation rules are given later in this section, and their implementation is provided in Appendix B. The LRP procedure is shown graphically in Fig. 4.

**Algorithm 2** LRP

---

```

 $a^{(0)} = x$ 
for  $l = 1 \dots L$  do
   $a^{(l)} = f_l(a^{(l-1)})$ 
end for
 $R^{(L)} = a^{(L)}$ 
for  $l = L \dots 1$  do
   $R^{(l-1)} = \text{relprop}(a^{(l-1)}, R^{(l)}, f_l)$ 
end for
return  $R^{(0)}$ 

```

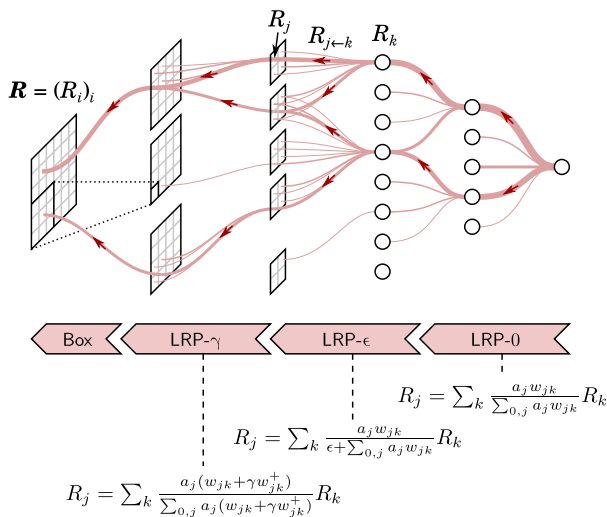
---

While LRP can, in principle, be performed in any forward computational graph, a class of neural networks, which is often encountered in practice, and for which LRP comes with efficient propagation rules that can be theoretically justified (see Section V) is deep rectifier networks [51]. The latter can be in large part abstracted as an interconnection of neurons of the type

$$a_k = \max\left(0, \sum_{0,j} a_j w_{jk}\right)$$

where  $a_j$  denotes some input activation, and  $w_{jk}$  is the weight connecting neuron  $j$  to neuron  $k$  in the layer above. The notation  $\sum_{0,j}$  indicates that we sum over all neurons  $j$  in the lower layer plus a bias term  $w_{0k}$  with  $a_0 = 1$ . For this class of networks, various propagation rules have been proposed (see Fig. 4). For example, the LRP- $\gamma$  rule [126] defined as

$$R_j = \sum_k \frac{a_j (w_{jk} + \gamma w_{jk}^+)}{\sum_{0,j} a_j (w_{jk} + \gamma w_{jk}^+)} R_k \quad (2)$$



**Fig. 4.** Illustration of the LRP propagation procedure applied to a neural network. The prediction at the output is propagated backward in the network, using various propagation rules, until the input features are reached. The propagation flow is shown in red.

redistributes based on the contribution of lower layer neurons to the given neuron activation, with a preference for positive contributions over negative contributions. This makes it particularly robust and suitable for the lower layer convolutions. Other propagation rules, such as LRP- $\epsilon$  or LRP-0, are suitable for other layers [126]. Additional propagation rules have been proposed for special layers, such as min/max pooling [13], [86], [127] and LSTM blocks [9], [11]. Furthermore, a number of other propagation techniques have been proposed [100], [170], [171], [202] with some of the rules overlapping with LRP for certain choices of parameters. For a technical overview of LRP, including a discussion of the various propagation rules and further recent heuristics, see [126].

An inspection of (2) shows an important property of LRP, of conserving relevance from layer to layer; in particular, we can show that, in the absence of bias terms,  $\sum_j R_j = \sum_k R_k$ . A further interesting property of this propagation rule is “smoothing”: consider the relevance can be written as  $R_j = a_j c_j$  and  $R_k = a_k c_k$  a product of activations and factors. Those factors can be directly related by the following equation:

$$c_j = \sum_k (w_{jk} + \gamma w_{jk}^+) \frac{\max(0, \sum_{0,j} a_j w_{jk})}{\sum_{0,j} a_j (w_{jk} + \gamma w_{jk}^+)} c_k. \quad (3)$$

This equation can be interpreted as a smooth variant of the chain rule for derivatives used for computing the neural network gradient [125]. Thus, analogous to SmoothGrad [176], LRP also performs some gradient smoothing; however, it embeds it tightly into the deep architecture so that only a single backward pass is required. In addition to smoothing, (3) can also be interpreted as a gradient that has been biased to positive values, an idea also found in methods, such as *DeconvNet* [201] or *Guided Backprop* [178]. This modified gradient view on LRP can also be leveraged to achieve a simpler and more general implementation of LRP based on “forward hooks,” which we describe in the second part of Appendix B, and which we use to apply LRP on VGG-16 [175] and ResNet-50 [65] in Section IV.

## E. Other Methods

We discuss, in this section, several other popular XAI approaches that either do not fall in the category of *post hoc* explanation approaches (and, therefore, are not covered in the sections above), that are specialized for a particular neural network architecture, or that make use of different units of interpretability than the input features.

In contrast to the discussed *post hoc* methods that apply to any DNN model, *self-explainable models* are designed from scratch with interpretability in mind. A self-explainable model can either be trained to solve an ML task directly from a supervised data set, or it can be used to approximate a black-box model on some representative input distribution. Examples of self-explainable models

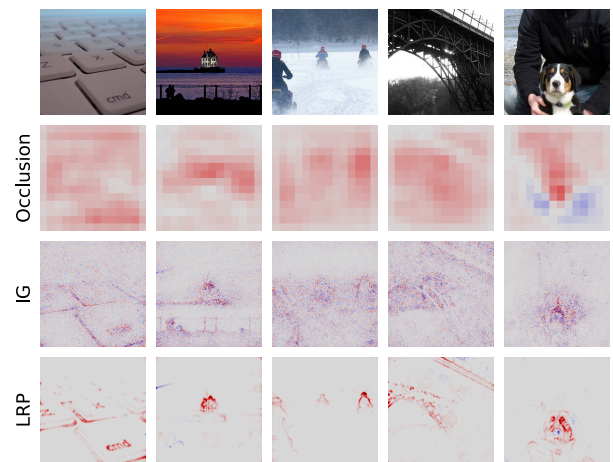
include simple linear models, or specific nonlinear models, e.g., neural networks with an explicit top-level summing structure [25], [28], [111], [143], [206]. In all of these models, each summand is linked only to one of a few input variables, which makes attribution of their prediction on the input variables straightforward. More complex architectures involving *attention mechanisms* were also proposed [15], [105], [195], and inspection of the attention mechanism itself can also deliver useful insights into the model prediction. While self-explainable models can be useful for many real-world tasks (a list of arguments in favor of these models can be found, e.g., in [154]), their applicability becomes more limited when the goal is to explain the strategy of some existing black-box model. In such a scenario, one would have to achieve the difficult task of closely replicating the black-box model for every possible input and perturbation of it while, at the same time, being constrained by the predefined interpretable structure.

Other methods are *specialized* for a particular deep neural network model for which generic explanation methods do not provide a direct solution. One such model is the graph neural network [91], [160], where the graph adjacency matrix given as input does not appear as it is usually the case in the first layer, but instead at every layer. Methods that have been proposed to explain these particular neural networks include the GNNExplainer [198] or GNN-LRP [162]. Other neural network architectures have a more conventional structure but still require a nontrivial adaptation of existing explanation methods, for example, extensions of LRP have been proposed to deal with the special LSTM blocks in recurrent neural networks [9], [11] or to handle attention units in the context of neural machine translation [38].

Further methods do not seek to explain in terms of input features but in terms of the latent space, where the directions in the latent space code for higher level concepts, such as color, material, object part, or object [17], [18], [205]. In particular, the TCAV method [89] produces a latent-space explanation for every individual prediction. Some techniques integrate multiple levels of abstraction (e.g., different layers of the neural networks) to arrive at a more informative explanation of the prediction process [162], [173], [203]. Finally, generative approaches have been proposed to build structured textual explanations of an ML model [67], [113].

#### IV. COMPARING EXPLANATION METHODS

The methods presented in Section III highlight the variety of approaches available for attributing the prediction of a deep neural network to its input features. This variety of techniques also translates into a variety of qualities of explanations. Illustrative examples of images and the explanation of predicted evidence for the ground-truth class as produced by the different explanation methods are shown in Fig. 5. The occlusion analysis is performed by



**Fig. 5.** Examples of images from ImageNet [157] with classes “space bar,” “beacon/lighthouse,” “snow mobile,” “viaduct,” and “greater swiss mountain dog.” Images are correctly predicted by the VGG-16 [175] neural network and shown along with an explanation of the predictions. Different explanation methods lead to different qualities of explanation.

occluding patches of size  $32 \times 32$  pixels with stride 16. Integrated gradients perform five integration steps starting from five random points near the origin in order to add smoothing (see Appendix A), resulting in 25 function evaluations. LRP explanations are obtained by applying the same LRP rules as in [126]. We observe the following qualitative properties of the explanations: occlusion-based explanations are coarse and are indicating relevant regions rather than the relevant pixel features. Integrated gradients produce very fine pixelwise explanations containing both substantial amounts of evidence in favor and against the prediction (red and blue pixels). LRP preserves the fine explanation structure but tends to produce less negative scores and attributes relevance to whole features rather than individual pixels.

In practice, it is important to reach an objective assessment of how good an explanation is. Unfortunately, evaluating explanations is made difficult by the fact that it is generally impossible to collect “ground-truth” explanations. Building such ground-truth explanations would, indeed, require the expert to understand how the deep neural network decides.

Standard ML models are usually evaluated by the utility (expected risk) of their decision behavior (e.g., [190]). Transposing this concept of maximizing utility to the domain of explanation, quantifying the utility of the explanation would first require to define what is the ultimate target task (the explanation being only an intermediate step) and then assessing by how much the use of explanation by the human increases its performance on the target task compared to not using it (see [14], [41], [62], and [159]). Because such end-to-end evaluation schemes are hard to set up in practice, general desiderata for

ML explanations have been proposed [123], [184]. Common ones include: 1) faithfulness/sufficiency; 2) human-interpretability; and 3) possibility to practically apply it to an ML model or an ML task (e.g., algorithmic efficiency of the explanation algorithm).

### A. Faithfulness/Sufficiency

A first desideratum of an explanation is to reliably and comprehensively represent the local decision structure of the analyzed ML model. A practical technique to assess such property of the model is “pixel-flipping” [158]. The pixel-flipping procedure tests whether removing the features highlighted by the explanation (as most relevant) leads to a strong decay of the network prediction abilities. The procedure is summarized in Algorithm 3.

---

#### Algorithm 3 Pixel-Flipping

---

```

pfcurve = []
for p in argsort(-R) do
     $x \leftarrow x - \{x_p\}$  (remove pixel  $p$  from the image).
    pfcurve.append( $f(x)$ ).
end for
return pfcurve

```

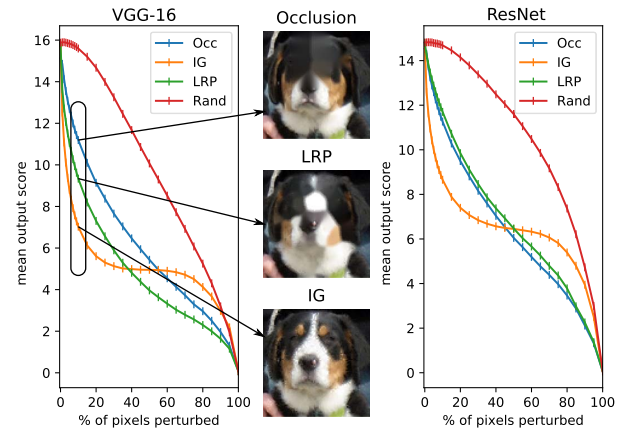
---

Pixel-flipping runs from the most to the least relevant input features, iteratively removing them and monitoring the evolution of the neural network output. The series of recorded decaying prediction scores can be plotted as a curve. The faster the curve decreases, the more faithful the explanation method is with respect to the decision of the neural network. The pixel-flipping curve can be computed for a single example or averaged over a whole data set in order to get a global estimate of the faithfulness of an explanation algorithm under study.

Fig. 6 applies pixel-flipping to the three considered explanation methods and on two models: VGG-16 [175] and ResNet-50 [65]. At each step of pixel-flipping, removed pixels are imputed using a simple inpainting algorithm, which avoids introducing visual artifacts in the image.

We observe that, for all explanation methods, removing relevant features quickly destroys class evidence. In particular, they perform much better than a random explanation baseline. Fine differences can, however, be observed between the methods: for example, LRP performs better on VGG-16 than on ResNet-50. This can be explained by VGG-16 having a more explicit structure (standard pooling operations for VGG-16 versus strided convolution for ResNet-50), which better supports the process of relevance propagation (see also [149] for a discussion of the effect of structure on the performance of explanation methods).

The second observation in Fig. 6 is that integrated gradients have by far the highest decay rate initially but stagnate in the later phase of the pixel-flipping procedure. The reason for this effect is that IG focuses on pixels to which the network is the most sensitive, without, however,



**Fig. 6. Pixel-flipping experiment for testing faithfulness of the explanation. We remove pixels found to be the most relevant by each explanation method and verify how quickly the output of the network decreases.**

being able to identify fully comprehensively the relevant pattern in the image. This effect is illustrated in Fig. 6 (middle) on a zoomed-in exemplary image of class “greater swiss mountain dog,” where the image after 10% flipping has lost most of its prediction score but visually appears almost intact. Effectively, IG has built an adversarial example [135], [185], i.e., an example whose visual content clearly disagrees with the prediction at the output of the network. We note that occlusion and LRP do not run into such adversarial examples. For these methods, pixel-flipping steadily and comprehensively removes features until class evidence has totally disappeared.

Overall, the pixel-flipping algorithm characterizes various aspects of the faithfulness of an explanation method. We note, however, that faithfulness of an explanation does not tell us how easy it will be for a human to make sense of that explanation. We address this other key requirement of an explanation in Section IV-B.

### B. Human Interpretability

Here, we discuss whether the presented explanation techniques deliver results that are meaningful to the human, i.e., whether the human can gain understanding into the classifier’s decision strategy from the explanation. Human interpretability is hard to define in general [123]. Different users may have different capabilities at reading explanations and at making sense of the features that support them [133], [147]. For example, the layman may wish for a visual interpretation, even approximate, whereas the expert may prefer an explanation supported by a larger vocabulary, including precise scientific or technical terms [14].

For the image classification setting, interpretability can be quantified in terms of the amount of information contained in the heatmap (e.g., as measured by the file size). An explanation with small associated file size is more likely to be interpretable by a human. The following table shows



average file sizes (in bytes<sup>1</sup>) associated with the various explanation techniques and for two neural networks.

|           | Occ          | IG     | LRP    |
|-----------|--------------|--------|--------|
| VGG-16    | <b>698.4</b> | 5795.0 | 1828.3 |
| ResNet-50 | <b>693.6</b> | 5978.0 | 2928.2 |

We observe that occlusion produces the lowest file size and is, therefore, the most “interpretable.” It, indeed, only presents to the user rough localization information without going into the details of which exact feature has supported the decision as done, e.g., by LRP. On the other side of the interpretability spectrum, we find integrated gradients. In the explanations this last method produces, every single pixel contains information, and this makes it clearly overwhelming to the human.

In practice, neural networks do not need to be explained in terms of input features. For example, the TCAV method [89] considers directional derivatives in the space of activations (where the directions correspond to higher level human-interpretable concepts) in place of the input gradient. Similar higher level interpretations are also possible using the occlusion and LRP methods, respectively, by perturbing groups of activations corresponding at a given layer to a certain concept, or by stopping the LRP procedure at the same layer and pooling scores on some group of neurons representing the desired concept.

### C. Applicability and Runtime

Faithfulness and interpretability do not fully characterize the overall usefulness of an explanation method. To characterize usefulness, we also need to determine whether the explanation method is applicable to a range of models that is sufficiently large to include the neural network model of interest and whether explanations can be obtained quickly with finite compute resources.

*Occlusion*-based explanations are the easiest to implement. These explanations can be obtained for any neural network, even those that are not differentiable. This also includes networks for which we do not have the source code and where we can only access their prediction through some online server. Technically, occlusion can, therefore, be used to understand the predictions of third-party models, such as <https://cloud.google.com/vision/> and <https://www.clarifai.com/models>. *Integrated gradients* require, instead for each prediction, an access to the neural network gradient. Given that most ML models are differentiable, this method is widely applicable also for neural networks with complex structures, such as ResNets [65] or SqueezeNets [79]. Integrated gradients are also easily implemented in state-of-the-art ML frameworks, such as PyTorch or TensorFlow, where we can make use of automatic differentiation. LRP assumes that the model is structured as (or can be converted to [85] and [86]) a neural network with a

canonical sequence of layers, for example, an alternation of linear/convolution layers, ReLU layers, and pooling layers. This stronger requirement and the implementation overhead caused by explicitly accessing the different layers (see Appendix B) will, however, be offset by the last characteristic that we consider in this section, which is the computational cost associated with producing the explanation. A runtime comparison<sup>2</sup> of the three explanation methods studied here is given in the following table (measured in *explanations per second*).

|           | Occ | IG  | LRP          |
|-----------|-----|-----|--------------|
| VGG-16    | 2.4 | 5.8 | <b>204.1</b> |
| ResNet-50 | 4.0 | 8.7 | <b>188.7</b> |

Occlusion is the slowest method as it requires reevaluating the function for each occluded patch. For image data, the runtime of occlusion increases quadratically with the step size, making the obtainment of high-resolution explanations with this method computationally prohibitive. Integrated gradients inherit pixelwise resolution from the gradient computation, which is  $O(1)$ , but require multiple iterations for the integration. The runtime is further increased if performing an additional loop of smoothing. LRP is the fastest method in our benchmark by an order of magnitude. The LRP runtime is only approximately three times higher than that of computing a single forward pass. This makes LRP particularly convenient for the large-scale analyses that we introduce in Section VI-D where an explanation needs to be produced for every single example in the data set.

## V. THEORETICAL FOUNDATIONS OF EXPLANATION METHODS

In parallel to developing explanation methods that address application requirements, such as faithfulness, interpretability, usability, and runtime, some works have focused on building theoretical foundations for the problem of explanation [116], [127] and establishing theoretical connections between the different methods [4], [128], [171].

Here, we present three frameworks: the Shapley values [116], [169], [179] that come from game theory, the Taylor expansions [13], [19], and the deep Taylor decomposition [127], which applies Taylor expansions repeatedly at each layer of a DNN. We then show how occlusion, integrated gradients, or LRP intersect for certain choices of parameters with these mathematical approaches.

### A. Shapley Values

The Shapley value [169] is a framework originally proposed in the context of game theory to determine individual contributions of a set of cooperating players  $\mathcal{P}$ .

<sup>1</sup>JPEG compression using the Pillow image processing library for python with a quality setting of 75/100 (standard settings).

<sup>2</sup>Explanations are computed in batches of (up to) 16 samples on a GPU and with explanation techniques implemented in PyTorch. Results are averaged over ten repetitions.

The method considers every subset of cooperating players  $S \subseteq \mathcal{P}$  and tests the effect of removing/adding the player  $i$  to  $S$  on the total payoff  $v(S)$  obtained by  $S$  if they cooperate. Specifically, Shapley values identify the contribution of player  $i$  to the overall coalition  $\mathcal{P}$  to be

$$\phi_i = \sum_{S \subseteq \mathcal{P} \setminus \{i\}} \alpha_S \cdot (v(S \cup \{i\}) - v(S))$$

where each subset  $S$  is weighted by the factor  $\alpha_S = |S|! \cdot (|\mathcal{P}| - 1 - |S|)! / |\mathcal{P}|!$ . Shapley values satisfy a number of axioms, in particular, efficiency ( $\sum_i \phi_i = v(\mathcal{P})$ ), symmetry, linearity, and zero added value of a dummy player. Shapley values are, in fact, the unique assignment strategy that jointly satisfies these axioms [169].

When transposing the method to the task of explaining an ML model [116], [179], the players of the cooperating game become the input features, and the payoff function becomes related to the DNN output. In [116], the payoff function is chosen to be the conditional expectation:  $v(S) = \mathbb{E}[f(\mathbf{x}) | \mathbf{x}_S]$ . Alternately, to make the score depend only on the model without assuming a specific input distribution, the payoff function can be set to

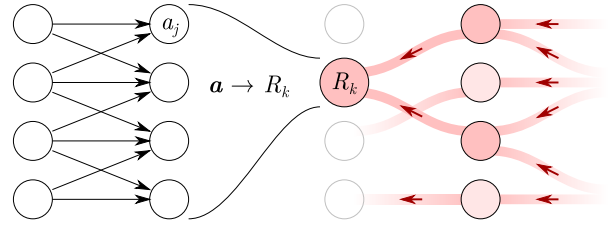
$$v(S) = f(\mathbf{x}_S) \quad (4)$$

i.e., input features not in  $S$  are set to zero (see [5]), and we will use this formulation to make connections to practical explanation methods in Section V-D. Note that Shapley values make almost no assumptions about the structure of the function  $f$  and can, therefore, serve as a general theoretical framework to analyze explanation methods. For specific functions, e.g., additive models of the type  $f(\mathbf{x}) = \sum_i f_i(x_i)$ , Shapley values (using (4)) take the simple form  $\phi_i = f_i(x_i)$ .

## B. Taylor Decomposition

Taylor expansions are a well-known mathematical framework to decompose a function into a series of terms associated with different degrees and combinations of input variables. Unlike Shapley values that evaluate the function  $f(\mathbf{x})$  multiple times, the Taylor expansion framework for explaining an ML model [13], [19], [42] evaluates the function once at some reference point  $\tilde{\mathbf{x}}$  and assigns feature contributions by locally extracting the gradient (and higher order derivatives). Specifically, the Taylor expansion of some smooth and differentiable function  $f: \mathbb{R}^d \rightarrow \mathbb{R}$  at some reference point  $\tilde{\mathbf{x}}$  is given by

$$\begin{aligned} f(\mathbf{x}) &= f(\tilde{\mathbf{x}}) \\ &+ \sum_i [\nabla f(\tilde{\mathbf{x}})]_i \cdot (x_i - \tilde{x}_i) \\ &+ \frac{1}{2} \sum_{ii'} [\nabla^2 f(\tilde{\mathbf{x}})]_{ii'} (x_i - \tilde{x}_i)(x_{i'} - \tilde{x}_{i'}) \\ &+ \dots \end{aligned}$$



**Fig. 7.** Graphical illustration of the function  $R_k(a)$  that DTD seeks to decompose on the input dimensions. Because  $R_k$  is complex, it is often replaced by an analytically more tractable model  $\hat{R}_k(a)$  that only depends on local activations.

where  $\nabla f$  and  $\nabla^2 f$  denote the gradient and the Hessian, respectively, and  $\dots$  denote the (nonexpanded) higher order terms. The zero-order term is the function value at the reference point and is zero if choosing a root point. There are as many first-order terms as there are dimensions, and each of them is bound to a particular input variable. Thus, they offer a natural way of attributing a function value  $f(\mathbf{x})$  onto individual linear components. There are as many second-order terms as there are pairs of ordered variables, and even more third-order and higher order terms. When the function is approximately locally linear, second and higher order terms can be ignored, and we get the following simple attribution scheme:

$$R_i = [\nabla f(\tilde{\mathbf{x}})]_i \cdot (x_i - \tilde{x}_i)$$

a product of the gradient and the input relative to our root point. In the general case, there are no closed-form approaches to find the root point, and it is instead obtained using an optimization technique.

## C. Deep Taylor Decomposition

An alternate way of formalizing the problem of attribution of a function onto input features is offered by the recent framework of deep Taylor decomposition (DTD) [127]. Deep Taylor decomposition assumes that the function is structured as a deep neural network and seeks to attribute the prediction onto input features by performing a Taylor decomposition at every neuron of each layer instead of directly on the whole neural network function. Deep Taylor decomposition assumes the output score has already been attributed onto some layer of activations  $(a_k)_k$ , and attribution scores are denoted by  $R_k$ . Deep Taylor decomposition then considers the function  $R_k(a)$ , where  $\mathbf{a} = (a_j)_j$  is the collection of neuron activations in the layer below. These quantities are illustrated in Fig. 7.

The function  $R_k(a)$  is typically very complex as it corresponds to a composition of multiple forward and backward computations. This function can, however, be approximated locally by some “relevance model”  $\hat{R}_k(a)$ , the choice of which will depend on the method that we have used for computing  $R_k$ . We then compute a Taylor expansion of this

**Table 1** Examples of Explanation Methods Applied With Different Parameters on Different Models and Whether They Can Be Embedded in Each of the Three Presented Theoretical Frameworks

|                                  | Shapley | Taylor | DTD |
|----------------------------------|---------|--------|-----|
| <i>Linear models</i>             |         |        |     |
| Occlude-1                        | ✓       | ✓      | (✓) |
| IG on Segment(0, $\mathbf{x}$ )  | ✓       | ✓      | (✓) |
| LRP-0                            | ✓       | ✓      | (✓) |
| <i>Nonlinear additive models</i> |         |        |     |
| Occlude-1                        | ✓       |        |     |
| <i>Deep rectifier networks</i>   |         |        |     |
| Occlude-1                        |         |        |     |
| IG on Segment(0, $\mathbf{x}$ )  |         | ✓      |     |
| LRP-0                            |         | ✓      | ✓   |
| LRP- $\epsilon/\gamma/\dots$     |         |        | ✓   |

function:

$$\begin{aligned}\hat{R}_k(\mathbf{a}) &= \hat{R}_k(\tilde{\mathbf{a}}) \\ &+ \sum_j [\nabla \hat{R}_k(\tilde{\mathbf{a}})]_j \cdot (a_j - \tilde{a}_j) \\ &+ \dots\end{aligned}$$

The linear terms define “messages”  $R_{j \leftarrow k}$  that can be redistributed to neurons in the lower layer, and messages received by a given neuron at a certain layer are summed to form a total relevance score

$$R_j = \sum_k [\nabla \hat{R}_k(\tilde{\mathbf{a}}^{(k)})]_j \cdot (a_j - \tilde{a}_j^{(k)}). \quad (5)$$

Here, we have added an index  $\{ \}^{(k)}$  to the root point to make explicit that different root points can be used for expanding different neurons. The redistribution procedure is iterated from the top layer toward the lower layers, until the input features are reached.

## D. Connections Between Explanation Methods

Having described the Shapley values and the simple and deep Taylor decomposition frameworks, we now present some results from the literature showing how some explanation methods reduce for certain choices of parameters to these frameworks. The different connections that we outline here are summarized in Table 1.

We start by connecting occlusion-based explanations of a linear model to Shapley values and Taylor decomposition.

**Proposition 1:** When applied to homogeneous linear models (of the type  $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$ ), occlusion with a patch size of 1 and a replacement value of 0 is equivalent to a Taylor decomposition with root point  $\tilde{\mathbf{x}} = \mathbf{0}$ , as well as Shapley values with value function given by (4).

The first connection is shown by the chain of equations  $f(\mathbf{x}) - f(\mathbf{x} - \{x_i\}) = w_i x_i = [\nabla f(\mathbf{0})]_i \cdot (x_i - 0)$ . For the Shapley values, we simply observe that  $\phi_i = \sum_{S \subseteq \mathcal{P} \setminus \{i\}} \alpha_S \cdot (w_i x_i) = 1 \cdot (w_i x_i)$ , which again

gives the same result. Integrated gradients and LRP-0 also yield the same result. Hence, for this simple linear model, all explanation methods behave consistently and in agreement with the existing theoretical frameworks. The connection of explanation methods to Shapley values for linear models was also made in [5].

The connection between integrated gradients and Taylor decomposition holds for a broader class of neural network functions, specifically deep rectifier networks (without biases):

**Proposition 2:** When applied to deep rectifier networks of the type  $f(\mathbf{x}) = \rho(W_L \rho(\dots \rho(W_2 \rho(W_1 \mathbf{x}))))$ , integrated gradients with integration path  $\{t\mathbf{x}; 0 < t \leq 1\}$  are equivalent to Taylor decomposition at  $\tilde{\mathbf{x}} = \epsilon \mathbf{x}$  in the limit  $\epsilon \rightarrow 0$ .

This can be shown by making the preliminary observation that a deep rectifier network is linear with constant gradient on the segment  $(0, \mathbf{x}]$  and then applying the chain of equations  $\int_\epsilon^1 x_i [\nabla f(t\mathbf{x})]_i dt = (1 - \epsilon) x_i [\nabla f(\epsilon \mathbf{x})]_i = [\nabla f(\epsilon \mathbf{x})]_i (x_i - \epsilon x_i)$ . This connection, along with the observation that a single gradient evaluation of a deep network can be noisy (see Section II-B), speaks against integrating on the segment  $(0, \mathbf{x}]$ . For this reason, we have opted in the experiments of Section IV to use a smoothed version of IG. A further result shows an equivalence between a “naïve” version of LRP (using LRP-0 at every layer) and Taylor decomposition.

**Proposition 3:** For deep rectifier nets of the type  $f(\mathbf{x}) = \rho(W_L \rho(\dots \rho(W_2 \rho(W_1 \mathbf{x}))))$ , applying LRP-0 at each layer is equivalent to a Taylor decomposition at  $\tilde{\mathbf{x}} = \epsilon \mathbf{x}$  in the limit  $\epsilon \rightarrow 0$ .

This result can be derived by taking the LRP formulation of (3) and setting  $\gamma = 0$ . This equation then reduces to

$$c_j = \sum_k w_{jk} \text{step} \left( \sum_{0,j} a_j w_{jk} \right) c_k$$

where  $\text{step}(t) = 1_{t>0}$ . This equation is exactly the same as the one that propagates gradients in a deep rectifier network. Hence, the input relevance computed by LRP becomes  $R_i = x_i c_i = x_i [\nabla f(\mathbf{x})]_i$  for which we have already shown the equivalence to simple Taylor decomposition in the proposition above. The connection has been originally made in [171].

**Proposition 4:** For deep rectifier networks of the type  $f(\mathbf{x}) = \rho(W_L \rho(\dots \rho(W_2 \rho(W_1 \mathbf{x}))))$ , applying LRP- $\gamma$  is equivalent to performing one step of deep Taylor decomposition and choosing the nearest root point on the line  $\{\mathbf{a} - t\mathbf{a} \odot (\mathbf{1} + \gamma \cdot \mathbf{1}_{w_k \geq 0}); t \in \mathbb{R}\}$ .

We choose the relevance model  $\hat{R}_k(\mathbf{a}) = a_k(\mathbf{a}) \cdot c_k$  with  $c_k$  constant (see [126] for a justification). Injecting the root point in the first-order terms of DTD (summands of (5)) gives

$$\begin{aligned}R_{j \leftarrow k} &= w_{jk} \cdot c_k \cdot (a_j - (a_j - t a_j \cdot (1 + \gamma \cdot 1_{w_{jk} \geq 0}))) \\ &= a_j \cdot (w_{jk} + \gamma w_{jk}^+) \cdot t \cdot c_k\end{aligned}$$

where  $t$  is resolved using the conservation equation  $\sum_j R_{j \leftarrow k} = R_k$ . LRP-0 is a special case of LRP- $\gamma$  with  $\gamma = 0$ . A similar procedure with another choice of reference point gives LRP- $\epsilon$  (see [126]).

## VI. EXTENDING EXPLANATIONS

The explanation methods that we have presented in the previous sections were applied to a particular class of models (deep neural networks) and produced particular types of explanations (attribution on the input features). We present, in the following, various extensions that broaden the applicability of these methods and diversify the type of explanation that can be produced. In particular, we will discuss: 1) higher order methods to produce richer explanations involving the combination of features; 2) a systematic way of extending explanation methods to non-neural network models, e.g., in unsupervised learning where explanations are also needed; 3) a principled way to ensure that explanations of DNN classifiers are class-discriminative; and 4) strategies to go beyond individual explanations to arrive at a general understanding of the ML model.

### A. Explaining Beyond Heatmaps

The locally linear structure of deep neural networks lends itself well to the heatmap-based methods that we have reviewed in this article, which we call first-order methods. However, special types of neural networks, e.g., that incorporate products between input or latent variables, lose that property. Neural networks with product structures commonly occur for relational tasks, such as comparing images [122] or collaborative filtering [66]. Graph neural networks [91], [160] multiply the input connectivity matrix multiple times and, consequently, also exhibit product structures. In that case, the neural network is no longer piecewise linear and typically becomes piecewise polynomial with its input.

For illustration, we present the BiLRP method [42], which assumes that we have a similarity model built as a dot product on some hidden representation  $\phi = \phi_L \circ \dots \circ \phi_1$  of a deep network

$$y(\mathbf{x}, \mathbf{x}') = \langle \phi_L \circ \dots \circ \phi_1(\mathbf{x}), \phi_L \circ \dots \circ \phi_1(\mathbf{x}') \rangle.$$

If all functions  $\phi_1, \dots, \phi_L$  are piecewise homogeneous linear, then  $y$  can be rewritten as a composition

$$y(\mathbf{x}, \mathbf{x}') = \psi_L \circ \dots \circ \psi_1(\mathbf{x}, \mathbf{x}')$$

with  $\psi_1, \dots, \psi_L$  piecewise bilinear. Using deep Taylor decomposition, but, at each step, applying a *second-order* Taylor expansion, we arrive conceptually at the attribution of the similarity score  $y(\mathbf{x}, \mathbf{x}')$  to *pairs* of input features. (Practically, the attribution can be expressed as a product of two branches of LRP computation, hence the



**Fig. 8.** Example of two images predicted to be similar, along with a BiLRP second-order attribution of their similarity score rendered as a bipartite graph. (figure is adapted from [42]). The explanation shows that the front part of the two planes jointly contributes to the predicted similarity.

name BiLRP) An example of BiLRP explanation for the similarity of two planes in VGG-16 feature space is shown in Fig. 8.

We observe that the explanation does not highlight individual features but instead pairs of features from the two input images, reflecting the fact that they are jointly relevant to explain the high similarity score.

Other higher order methods include GNN-LRP that explains graph neural networks in terms of collections of edges [162] or integrated Hessians [81], which can be seen as the second-order extension of integrated gradients.

Finally, we note that the Shapley framework also offers the possibility to quantify the joint contribution of two interacting players through the Shapley interaction index [55]. Lundberg *et al.* [115] recently applied this concept to compute higher order explanations of an ML model. Here, the interaction value between feature  $i$  and feature  $j$  can be computed as

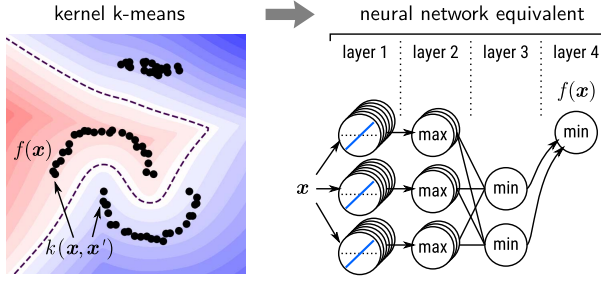
$$\phi_{ij} = \sum_{S \subseteq \mathcal{P} \setminus \{i, j\}} \alpha_S \cdot (v(S \cup \{i, j\}) - v(S \cup \{i\}) - v(S \cup \{j\}) + v(S))$$

with the weight factor  $\alpha_S = |\mathcal{S}|! \cdot (|\mathcal{P}| - 2 - |\mathcal{S}|)! / 2^{|\mathcal{P}| - 1}!$ . Shapley interaction values allow for a separate consideration of interaction effects, which can be crucial for understanding the model prediction strategy (see Section VII-C for a worked-through example in a medical application). However, although theoretically valid, these Shapley interaction terms are, in most cases, infeasible to compute for complex ML models, such as deep neural networks, and approximations are, therefore, required [120].

### B. Explaining Beyond Deep Networks

Deep neural networks have been shown to perform extremely well on classification or regression tasks. However, for unsupervised problems, such as anomaly detection or clustering, although deep models have also reached successes [27], [155], [156], shallow models (e.g., centroid-based [119], PCA-based [64], kernel-based [141], or combinations of them [37], [72], [164])





**Fig. 9.** Left: kernel k-means applied to a toy 2-D problem with three clusters. Red and blue in the background represent the positive and negative values of the logit function for a given cluster. Right: four-layer neural network equivalent of the kernel k-means logit score [85].

remain popular workhorses. As these models are not given in the form of a neural network, a direct application of methods designed in the context of linear models and DNNs is not feasible.

While a possible approach to explaining would be to train a surrogate neural network to match the output of the unsupervised model, it was found that some unsupervised models, such as k-means clustering [119] or kernel density estimation [141], can be directly rewritten as a neural network (or “neuralized”), without requiring any retraining or architecture search [85], [86].

Consider, for illustration, a kernel k-means model of the type studied in [37]. For this type of model, and assuming a Gaussian kernel  $\mathbb{K}(x, x') = \exp(-\gamma \|x - x'\|^2)$ , the probability ratio in favor of a given cluster  $\omega_c$  can be expressed as

$$\frac{P(\omega_c|x)}{1 - P(\omega_c|x)} = \frac{(Z_c^{-1} \sum_{i \in C_c} \mathbb{K}(x, x_i))^{\beta/\gamma}}{\sum_{k \neq c} (Z_k^{-1} \sum_{j \in C_k} \mathbb{K}(x, x_j))^{\beta/\gamma}}. \quad (6)$$

This is a power-assignment model applied to the kernel density functions of each cluster. The sets  $C_c$  and  $C_k$  are the representatives for clusters  $c$  and  $k$ , and  $Z_c$  and  $Z_k$  are respective normalization factors. An example of decision function produced by this model for a three-cluster problem is shown in Fig. 9 (left). It is clear that (6) is *a priori* not composed of neurons. However, it can be reorganized into the following sequence of detection and pooling functions [85]:

$$\log \left[ \frac{P(\omega_c|x)}{1 - P(\omega_c|x)} \right] = \beta \min_{k \neq c}^\beta \left\{ \min_{j \in C_k}^\gamma \left\{ \max_{i \in C_c}^\gamma \left\{ w_{ij}^\top x + b_{ijk} \right\} \right\} \right\}$$

with  $w_{ij} = 2(x_i - x_j)$  and  $b_{ijk} = \|x_j\|^2 - \|x_i\|^2 + \gamma^{-1}(\log Z_k - \log Z_c)$  are parameters of the first linear layer. This layer is followed by a hierarchy of log-sum-exp computations interpretable as canonical max- and min-pooling operations. The neuralized version of kernel k-means is depicted in Fig. 9 (right).

With this structure, explanation techniques, such as LRP can be used to produce explanations of cluster membership [85]. In particular, the LRP backward pass first identifies the most relevant class competitors, then the most relevant representatives (i.e., data points) of these class competitors and of the cluster of interest, and, finally, the most relevant directions in input space, thereby producing a heatmap-based explanation of the cluster assignment [85].

### C. Explaining Beyond Output Neurons

The concept of neuralization can also be applied in a supervised learning setting, for improving the explanation of a deep neural network classifier. So far, we have explained quantities at the output of the last linear layer of the network. Because these output quantities are unnormalized, they may respond positively to several classes, thereby lacking selectivity. The problem of class selectivity was highlighted e.g., in [57], [80], [126] along with practical solutions to overcome this effect. Here, we present the “neuralization” approach of [126], which first makes the observation that ratios of probabilities as given by the top-layer soft-assignment model can be expressed as

$$\frac{P(\omega_c|x)}{1 - P(\omega_c|x)} = \frac{\exp(w_c^\top a)}{\sum_{k \neq c} \exp(w_k^\top a)}.$$

This computation can then be reorganized in the two-layer neural network

$$\log \left[ \frac{P(\omega_c|x)}{1 - P(\omega_c|x)} \right] = \min_{k \neq c} \{ (w_c - w_k)^\top a \}$$

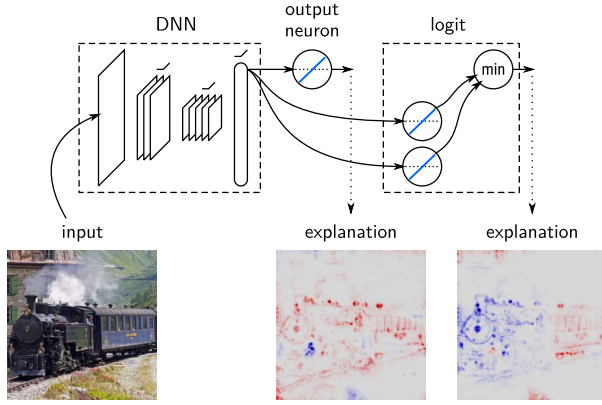
where min is a soft minimum implemented by a log-sum-exp computation. The DNN processing up to the output neuron or up to the output of the neuralized logit model is illustrated in Fig. 10 along with LRP explanations for these two quantities associated with the class “passenger\_car.”

In the first explanation, both the passenger car and the locomotive can be seen to contribute. In the second explanation, the locomotive turns blue. The latter is, indeed, speaking for the class locomotive, which mechanistically lowers the probability for the class “passenger\_car” [126]. This example shows that it is important in the presence of correlated features to precisely define what quantity (unnormalized score or logit) we would like to explain.

We note that while neuralization has served here to support LRP-type explanations, the concept could potentially be used for other purposes. The identified neural network structure may help to gain further understanding of the model or provide intermediate representations that are potentially useful to solve related tasks.

### D. Explaining Beyond Individual Predictions

In practice, we may not only be interested in explaining how the DNN predicts a single data point but also in the



**Fig. 10.** Deep neural network to which we append a neuralized version of the log-likelihood ratio [126]. Considering the latter quantity instead of the DNN output leads to a different explanation.

statistics of them for a whole data set. This may be useful to validate the model in a more complete manner. Let  $f : \mathbb{R}^d \rightarrow \mathbb{R}$  be a function that takes a data point as input and predicts evidence for a certain class for each data point. Consider a data set  $x_1, \dots, x_N$  of such data points. The total class evidence can be represented as a function  $g : \mathbb{R}^{N \times d} \rightarrow \mathbb{R}$ , where

$$g(x_1, \dots, x_N) = \sum_{n=1}^N f(x_n).$$

This composition of the neural network output and a sum-pooling remains explainable by all methods surveyed here; however, the explanation is now high-dimensional ( $N \times d$ ).

1) *Relevance Pooling*: Practically, we may be not be interested in explaining every single data point in terms of every single input features. A more relevant information to the user would be the overall contribution of a subgroup of features  $\mathcal{I}$  on a group of data points  $\mathcal{G}$  (see [101] and [128]). In particular, the integrated gradient and LRP methods surveyed here produce explanations that satisfy the conservation property

$$g(x_1, \dots, x_N) \approx \sum_{n=1}^N \sum_{i=1}^d R_{i,n}$$

and that can be converted to a coarse-grained explanation

$$\approx \sum_{\mathcal{G}} \sum_{\mathcal{I}} \underbrace{\sum_{n \in \mathcal{G}} \sum_{i \in \mathcal{I}} R_{i,n}}_{R_{\mathcal{I}, \mathcal{G}}}$$

which still satisfies the desired conservation property. As an illustration of the concept, we consider the “concrete compression strength” example of Section II. Data points

are grouped in three k-means clusters, and features are grouped in two sets: the singleton {age} and the set of all remaining features describing concrete composition. The pooled analysis is illustrated in Fig. 11.

This analysis gives further insight into our predictive model. We observe that most distinguishing factors, especially age, contribute negatively to strength. In other words, a “typical” age and composition is a recipe for strength, whereas high/low values tend to be explanatory for weakness. Notably, one data cluster stands out by having composition features that are explanatory for strength.

2) *Spectral Relevance Analysis (SpRAY)* [104]: While, in Section VI-D1, we have reduced the dimensionality through pooling, other analyses are possible. For example, the SpRAY method [104] does not assume a fixed pooling structure (e.g., a partition of data points and a partition of features) and applies instead a clustering of explanations in order to identify prototypical decision strategies. Algorithm 4 outlines the three steps procedure used by SpRAY:

---

**Algorithm 4** SpRAY

---

**for**  $n = 1$  **to**  $N$  **do**

$R^{(n)} \leftarrow \text{explain}(x^{(n)}, f)$

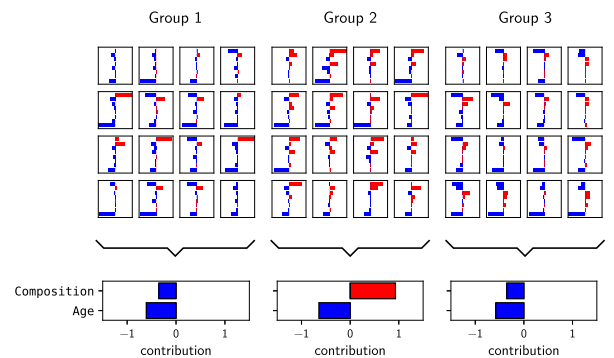
$\bar{R}^{(n)} \leftarrow \text{normalize}(R^{(n)})$

**end for**

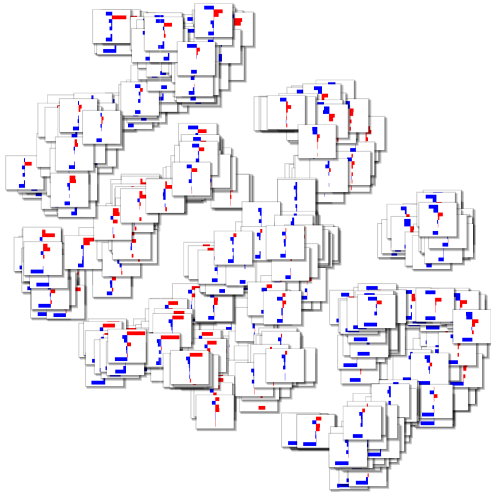
$\text{clustering}(\{\bar{R}^{(1)}, \dots, \bar{R}^{(N)}\})$

---

The method first produces an explanation for each data point. In principle, any explanation method can be used, e.g., occlusion, integrated gradients, or LRP. Explanations are then normalized (e.g., blurred and standardized) to become invariant to small pixelwise or saliency variations. Finally, a clustering algorithm is applied to the normalized explanation, and examples with the same cluster index can be understood as being associated with some prototypical



**Fig. 11.** Pooled analysis. Top: featurewise contributions for the prediction on three clusters of the Concrete Compressive Strength Data set [197]. Bottom: coarse-grained explanations obtained by pooling contributions on data clusters and groups of features.



**Fig. 12.** *SpRAY analysis. Explanation of the predictions of the Concrete Compressive Strength Data set [197], displayed at coordinates corresponding to their t-SNE embedding. This analysis provides a visualization of the overall classifier's strategy.*

decision strategy, e.g., looking at the object and looking at the background. Alternately, the clustering step can be replaced by a low-dimensional embedding step to produce a visual map of the overall decision structure of the ML model. The SpRAY analysis is illustrated in Fig. 12 on the same Concrete Compressive Strength Data set [197] used before.

Here, we observe, for example, that a typical decision strategy (bottom left) consists of explaining high compressive strength based on the first feature (high cement concentration), whereas another typical decision strategy (bottom right) consists of predicting low compressive strength based on the last feature (low concrete's age). This rich visual feedback can serve to validate the prediction strategy with expert knowledge to make sure that the model does not predict based on a data set artifact. We will present, in Section VIII-A, a successful real-world use case of the SpRAY analysis for inspecting a state-of-the-art model trained on a large image classification data set.

Altogether, relevance pooling and SpRAY support a variety of data set-wide analyses that are useful to explore and characterize the decision strategies of complex models trained on large data sets.

## VII. WORKED-THROUGH EXAMPLES

In this article, we have motivated the use of explanation in the context of deep learning models and showcased some methods for obtaining explanations. Here, we aim to take a practical look for the user to assess when the explanation is required, what are common issues with applying explanation techniques/setting their hyperparameters, and, finally, how to make sure that the produced explanations deliver meaningful insights for the human.

### A. Example 1: Validating a Face Classifier

In the first worked-through example, we wish to train an accurate classifier for predicting a person's age from images of faces. We will show how to use an explanation for this task, in particular, to verify that the model is not using "wrong" features for its decisions.

Let us use for this the Adience benchmark data set [44], providing 26 580 images captured "in the wild" and labeled into eight ordinal groups of age ranges  $\{(0-2), (4-6), (8-13), (15-20), (25-32), (38-43), (48-53), (60+)\}$ .

Because the number of examples in this data set is limited and likely not sufficient to extract good visual features, we adopt the common approach of starting with a generic pretrained classifier and fine-tune it on our task. We download a VGG-16 [175] neural network architecture pretrained on ImageNet [35] obtainable from [modelzoo.co](http://modelzoo.co). First, test results after training using stochastic gradient descent (SGD) [108] report reasonable performance, with *exact* and *one-off* [44], [168] prediction accuracy<sup>3</sup> of 56.5% and 90.0%, respectively. Here, the one-off accuracy considers predictions of (up to) one age group away from the true label as correct.

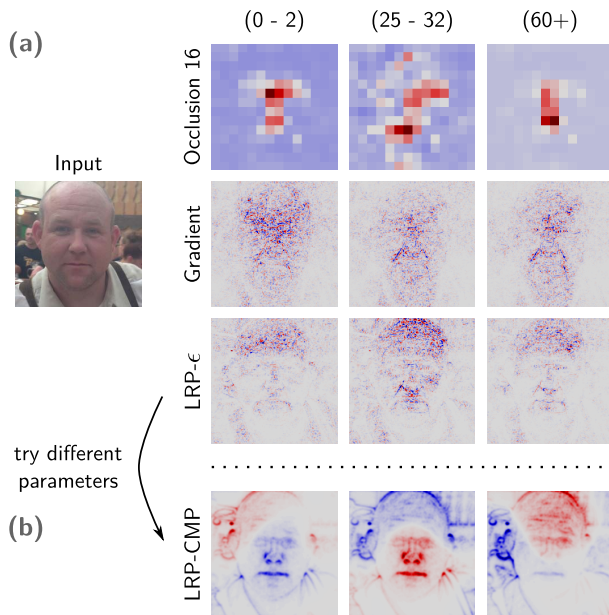
In order to understand the learned prediction strategies of our model and to verify that it uses meaningful features in the training data, we test different explanation methods, first, a simple occlusion and gradient analysis, and we also take an off-the-shelf explanation software, the LRP Toolbox [102] for Caffe [83], and choose the method LRP configured to perform "LRP- $\epsilon$ " on all layers in the first attempt. Explanations are shown for a given image in Fig. 13(a).

Some insight can be readily obtained from these explanations, e.g., the classifier has learned to attend the center of the image where the face is located. However, we also observe several limitations of the explanations: here, the occlusion-based analysis produces a blue background (negatively relevant) instead of identifying the background as irrelevant. The next two explanations (gradient and LRP- $\epsilon$ ) are, here, overly complex with frequent local sign changes, making it hard to extract further insights, especially what are the features that contribute to different age groups. The reason for such poor performance is that we have either not considered a broad enough spectrum of explanation methods, or the parameters of these methods have not been set optimally. This leads to our first recommendation:

**Try different parameters of the explanation techniques**

Specifically, we will now try an alternate LRP preset called "LRP-CMP" that applies a composite strategy [94], [103], [126] where different rules are applied at different layers. Explanations obtained with this new rule are

<sup>3</sup>Results have been averaged over the official preselected fivefold data set split [44].



**Fig. 13.** Top: explanations produced by different methods on the VGG-16 age model. Results are shown for the output neurons associated with age group labels (0-2), (25-32), and (60+), respectively. Bottom: application of the layer-dependent LRP-CMP decomposition strategy.

given in Fig. 13 (bottom). The new explanations highlight features in a much more interpretable way, and we also start to better understand what speaks—according to the model—in favor of or against certain age groups. For example, explanations amusingly reveal baldness as a feature corresponding to both age groups (0-2) and (60+). In the shown sample, baldness contributes evidence for the classes (0-2) and (60+), while it speaks against the age group (25-32). Relatedly, the expression of the man’s chin and mouth area contradicts class (0-2) more than class (60+) but “looks like” it would belong to a person aged (25-32).

Let us now move back to the initial question, namely, how to verify that the model is using the right features for predicting. While the decision structure of the model was meaningful in Fig. 13, we would like to verify that it is also the case for other test cases. Fig. 14 (top) shows further examples from the Adience data set, a woman labeled (60+), and three images of the same male labeled (48-53) with smiles of varying intensities.

We apply LRP with the same preset “LRP-CMP” on these images. LRP evidence for each image for the class (60+) is shown in Fig. 14 (middle). Surprisingly, according to the model, broad smiling contradicts the prediction of belonging to the age group (60+). Smiling is, however, clearly a confounding factor, which reliably predicts the age group only to the extent that no such case is present in the training data. This predicting strategy is related to

the “Clever Hans”<sup>4</sup> effect [104], and we can, therefore, formulate our second recommendation:

#### Unmask “Clever Hans” examples

Note that instead of screening through all images manually, we can also use techniques, such as SpRAy [104], which perform such analysis semiautomatically for large data sets, such as ImageNet (see Section VIII-A for successful applications).

While, for the examples showcased in Fig. 14, other features may compensate for the “smiling” effect, here, almost all features other than the smile also affect the decision toward this age group positively, and this will cause errors for less clear-cut cases. This may explain why the accuracy of the ImageNet-based model is not very high and can point at the fact that the test set accuracy may drop dramatically on new data sets, e.g., comprising more old people smiling.

Instead, we would like our model to be robust to a subject’s mood when predicting his or her age. We, thus, need to find a way to prevent Clever Hans effects, e.g., prevent the model to associate smiling with age. One reason the model has learned that connection in the first place is the extreme population imbalance among the age groups of the Adience data set, a problem that is shared with many other data sets of face images, e.g., [78], [153]. We, therefore, add the second pre-training phase in between the ImageNet initialization and the *actual* training based on the Adience data,

<sup>4</sup>“Clever Hans” was a famous horse at the beginning of the 20th century, which was believed by his trainer to be capable of performing arithmetic calculations. Subsequent analyses revealed that the horse was not performing arithmetic calculations but was detecting cues on the face of his trainer to produce the right answers. In ML, the term “Clever Hans” can be used to designate strategies that mimic the expected behavior but are based on unexpected correlations or artifacts in the data [104].



**Fig. 14.** LRP heatmaps demonstrating the effects of ImageNet [35] pretraining (middle) compared to additional IMDB-WIKI [153] pretraining (bottom). All heatmaps show the model decision with respect to age group (60+).



by using the considerably larger IMDB-WIKI [153] data set. The IMDB-WIKI data set consists of 523 051 images from 20 284 celebrities (460 723 images from the Internet Movie Database (IMDB) and 62 328 images from Wikipedia) at different ages, labeled with 101 labels (0–100 years, one label per year). The IMDB-WIKI data set also suffers from highly imbalanced label populations. However, we follow [153] and renormalize the age distribution by undersampling the more frequent classes until approximately 260 000 samples are selected overall. Furthermore, we assume that, since the IMDB-WIKI data set is composed of photos of public figures (taken at publicized events), the ratio of expressed smiles in higher age groups will be more frequent than in the Adience data set, which has been captured “in the wild”. A comparison of performance on the Adience benchmark of the original model (pretrained on ImageNet only) and the improved model is given in following the table.

|                      | accuracy | 1-off |
|----------------------|----------|-------|
| ImageNet pretrained  | 56.5     | 90.0  |
| IMDB-WIKI pretrained | 63.0     | 96.0  |

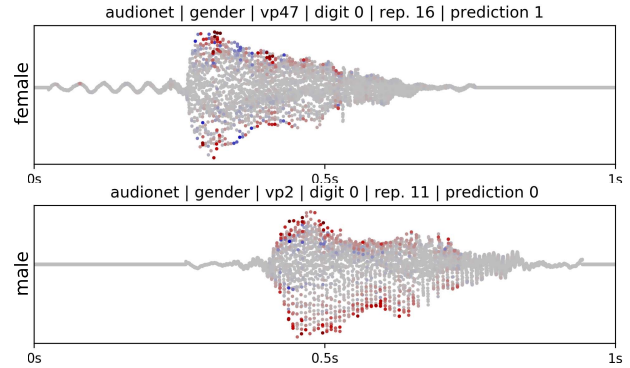
We observe that the additional and more domain-specific IMDB-WIKI pretraining step has improved the generalization performance of the VGG-16 model. Furthermore, we will see that it also prevented the model from associating smiling exclusively with younger age groups. Fig. 14 (bottom) shows LRP heatmaps for all four examples and age label (60+). For the woman, the model has shifted its attention from the hair and clothes to the face region and neck, and no longer considers the smile as contradictory to the class. A similar effect can be observed for the samples showing the male person. The model’s age prediction capabilities can no longer be fooled by just smiling into the camera. However, by introducing the IMDB-WIKI pretraining step, we have apparently replaced the smile-related Clever Hans strategy with another one, related to the presence of glasses in images of males in higher age groups. This leads to our third recommendation:

#### Iteratively validate and improve the model

We can do so until the model solely relies on its predictions on meaningful face features. For that, choosing a better pretraining may not be sufficient, and other more advanced interventions may be required.

### B. Example 2: Identifying Male and Female Speech Features

After demonstrating how explanations can be used to unmask Clever Hans strategies or more generally validate a classifier, we will now discuss another use case, where explanations are, this time, applied not to get a better model but to gain new (scientific) insights. In this worked-through example, we will show that explanations can be used to identify male and female features in speech.

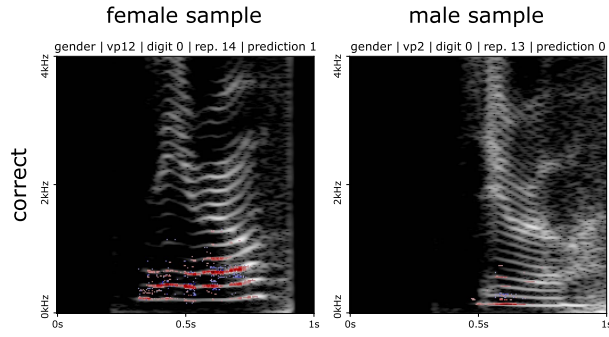


**Fig. 15.** Explanations based on waveform representation of speech data. Correct prediction of female (top) and male (bottom) subjects. The waveform data are visualized as a scatter plot of 8000 discrete measurements, color-coded according to relevance attribution for the true class label.

Before going into the analysis, let us first introduce the data and the model used for the speaker’s male versus female classification task. As training data, we use the recently recorded AudioMNIST [20] data set, comprised of 30 000 audio recordings of spoken digits from 60 different speakers, with 50 repetitions per digit and speaker, in a 48-kHz sampling frequency. Next to annotations for a spoken digit (0–9) and sex of speaker (48 male, 12 female), the data set provides labels for speaker age, accent, and origin. We begin by training a deep neural network model on the raw waveform data, which is first downsampled to 8 kHz, and randomly padded with zeroes before and after the recorded signal to obtain a 8000-D input vector per sample. A CNN architecture comprised of six 1-D-convolution layers interleaved with max-pooling layers and topped off with three fully connected layers [20] and ReLU activation units after each weighted layer is prepared for optimization. In order to prevent the model from overfitting on the more frequent population of samples labeled as “male”, we (randomly) select 12 speakers from both classes. The model is then trained and evaluated in a fourfold cross-validation setting, in which the 24 speakers are grouped into four sets of three male speakers and three female speakers. Each of the four splits, thus, contains 1000 waveform features. Two folds are used for training, while one of the remaining data splits is reserved for validation and testing. The model reaches an average test set accuracy ( $\pm$  standard deviation) of  $91.74\% \pm 8.60\%$  across all splits.

With the goal of understanding the data better by explaining the model, we consider two examples predicted by the network to be male and female and apply LRP to visualize those predictions. Here, the waveform is represented as a scatter plot where each time step is color-coded by its relevance. Results are shown in Fig. 15.

The explanations reveal that the model predominantly uses the *outer hull* of the waveform signal for decision-making. For a human observer, however, these



**Fig. 16.** Left: spectrogram representation of digit “zero” spoken by female speaker “vp12.” Right: spectrogram representation of digit “zero” spoken by male speaker “vp2.” Relevance maps are shown with respect to the samples’ true classes.

explanations are difficult to interpret due to the limited accessibility of the data representation in the first place (see Fig. 15). Although the model performs reasonably well on waveform data, it is hard to obtain a deeper understanding beyond the network’s *modus operandi* based on relevance maps due to the limitations imposed by the data representation itself. We, therefore, opt to change the data representation for improved interpretability.

#### Make your input features interpretable

More precisely, we exchange the raw waveform representation of the data with a corresponding  $228 \times 230$  (time  $\times$  frequency) shaped spectrogram representation by applying a short-time Fourier transform (time segment length of 455 samples, with 420 samples overlap), cropped to a  $227 \times 227$  matrix by discarding the highest frequency bin and the last three time segments. Consequently, we also exchange the neural network architecture and use an AlexNet [99] model, which is able to process the transformed input data using 2-D-convolution operators.

Fig. 16 visualizes two input spectrograms, with corresponding relevance maps (only relevance values with more than 10% relative amplitude) drawn on top.

Heatmap visualizations based on spectrogram input data are more informative than those for waveform data and reveal that the model has learned to distinguish between male and female speakers based on the lowest fundamental frequencies [male speakers; see Fig. 16 (right)], and immediate harmonics [female speakers; see Fig. 16 (left)] shown in the spectrogram. Many incorrectly classified samples with the ground-truth label “male” show large gaps between frequency bands often occurring in samples from female speakers. Note that these insights are consistent with the literature [187].

#### Gain insights by explaining predictions

As a noteworthy side effect, the increase in interpretability from switching from a waveform data representation to spectrogram data representation does *not* come at a

price of model performance. On the contrary, model performance is even increased slightly from 91.74% to 95.87%.

For applications where a change of input representation into images is not possible, e.g., because the training data are not available or because the human-interpretable image representation does not allow for training an accurate model, we can try to increase interpretability by different means. For our speech example, that could consist of segmenting the time series into meaningful phases, as done in the ECG analysis, and compute relevance with respect to these phases rather than the individual time points. Furthermore, the data and the explanations can be projected in different domains (e.g., similar to the TCAV approach [89]). In our speech example, projecting the time series and the explanations on a Fourier basis would, for example, allow for understanding frequencies that really matter for the decision. All this “postprocessing” of the explanation can help to understand the results even though the input features are hardly interpretable.

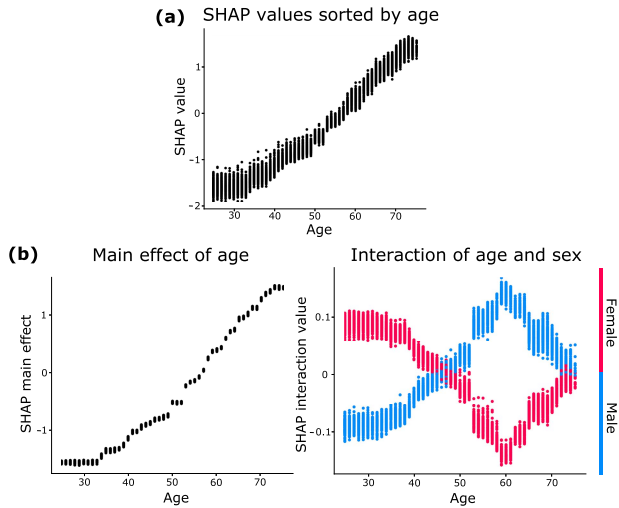
### C. Example 3: Identifying Interacting Mortality Risk Factors

The third worked-through example demonstrates possible limitations of first-order explanation methods and the need for a detailed (higher order) analysis of explanation results, especially when aiming to draw causal conclusions on the basis of the explanations. The example is adapted from the work of Lundberg *et al.* [115]. It concerns a medical application, in particular, a statistical model used for predicting mortality called “cox proportional hazards model” [33], fit on data from NHANES I with follow-up mortality data from the NHANES I Epidemiologic Follow-up Study [32]. The mortality data set consists of 14 407 individuals and 79 features, and the authors use a tree-based model<sup>5</sup> and a Shapley additive explanation (SHAP) method.

Fig. 17(a) shows the computed SHAP values for one of the considered features, namely, the age of the participants. Consistent with common medical knowledge, the SHAP values indicate a higher mortality risk for people with higher age. The vertical dispersion at a single age value results from interaction effects in the model, which is hidden when only considering the first-order explanations. In this data set, the sex of the participant modulates the risk associated with the factor “age”. This interesting relation can be derived from the SHAP interaction values.

Fig. 17(b) shows the main effect of age (left plot) and the interaction effect between age and sex (right plot) for the risk of mortality. These values constitute the diagonal and off-diagonal elements of the SHAP interaction matrix, respectively. One can see that age is on its own a very strong risk factor; however, depending on the sex of the person, this risk factor is modulated differently over different age groups. While, at a young age, the mortality risk is

<sup>5</sup>Lundberg *et al.* [115] also propose a polynomial-time algorithm for computing the Shapley value explanations on tree-based models.



**Fig. 17. Identifying mortality risk factors from explanations.** (a) First-order SHAP values show that high age correlates with high mortality risk. The impact of other interacting factors on the age contribution to risk remains hidden. (b) The SHAP interaction matrix can be decomposed into the main effects (diagonal elements) and the interaction effects (off-diagonal elements). Here, the interaction effect shows that the risk factor “age” is largely modulated by the sex of the participant (figure is adapted from [115]).

slightly higher for women, it reverses and is much higher for men at age 60 and, finally, reaches the same level for the high age group. This example, therefore, shows that, by only considering first-level explanations, we may miss important details and arrive at incomplete conclusions. Therefore, we recommend whenever possible to look at these interactions.

#### Look at interactions to get deeper insights

We note that a general limitation of higher order explanations is that they need more data to be extracted with statistical significance. This is not a problem when considering models taking few input features and trained on large data sets, but this can become more challenging for high-dimensional data, and in that case, only first-order effects can be captured robustly. Furthermore, even with interaction effects, performing causal inference, e.g., deriving mortality risk factors from epidemiological data, remains difficult due to unobserved effects, such as collider or measurement bias [69].

## VIII. SUCCESSFUL USES OF EXPLANATION TECHNIQUES

Interpretation methods can be applied for a variety of purposes. Some works have aimed to understand the model’s prediction strategies, e.g., in order to validate the model [104]. Others visualize the learned representations and try to make the model itself more interpretable [75]. Finally, other works have sought to use explanations to learn about the data, e.g., by visualizing interesting input-prediction patterns extracted by a deep neural network model in scientific applications [186].

Technically, explanation methods have been applied to a broad range of models ranging from simple bag-of-words-type classifiers or logistic regression [13], [28] to feedforward or recurrent deep neural networks [9], [11], [13], [171] and, more recently, also to unsupervised learning models [85], [86]. At the same time, these methods were able to handle different types of data, including images [13], speech [20], text [10], [38], and structured data, such as molecules [162], [165] or genetic sequences [191].

Some of the first successes in interpreting deep neural networks have occurred in the context of image classification, where deep convolutional networks have also demonstrated very high predictive performance [65], [99]. Explanation methods have for the first time allowed to open these “black boxes” and obtain insights into what the models have actually learned and how they arrive at their predictions. For instance, the works [134], [174]—also known in this context as “deep dreams”—highlighted surprising effects when analyzing the inner behavior of deep image classification models by synthesizing meaningful preferred stimuli. They report that the preferred stimuli for the class “dumbbell” would, indeed, contain a visual rendering of a dumbbell, but the latter would systematically come with an arm attached to it [131], demonstrating that the output neurons do not only fire for the object of interest but also for correlated features.

Another surprising finding was reported in [101]. Here, interpretability—more precisely, the ability to determine which pixels are being used for prediction—helped to reveal that the best performing ML model in a prestigious international competition, namely, the PASCAL visual object classification (VOC) challenge, was actually relying partly on artifacts. The high performance of the model on the class “horse” could, indeed, be attributed to detecting a copyright tag present in the bottom left corner of many horse images of the data set,<sup>6</sup> rather than detecting the actual horse in the image. Other effects of similar type have been reported for other classes and data sets in many other works, e.g., in [147], models were shown to distinguish between the class “Husky” and “Wolf” solely based on the presence or absence of snow in the background.

These discoveries have been made rather accidentally by researchers carefully analyzing suspicious explanations. It is clear that such laborious manual inspection of heatmaps does not scale to big data sets with *millions* of examples. Therefore, systematic approaches to the interpretation of ML models have recently gained increased attention.

### A. From Explanations to Understanding Large ML Models

This section describes two examples of a systematic analysis of a large number of heatmaps. In the first case,

<sup>6</sup>The presence of these artifacts in the benchmark data set had gone unnoticed for almost a decade.

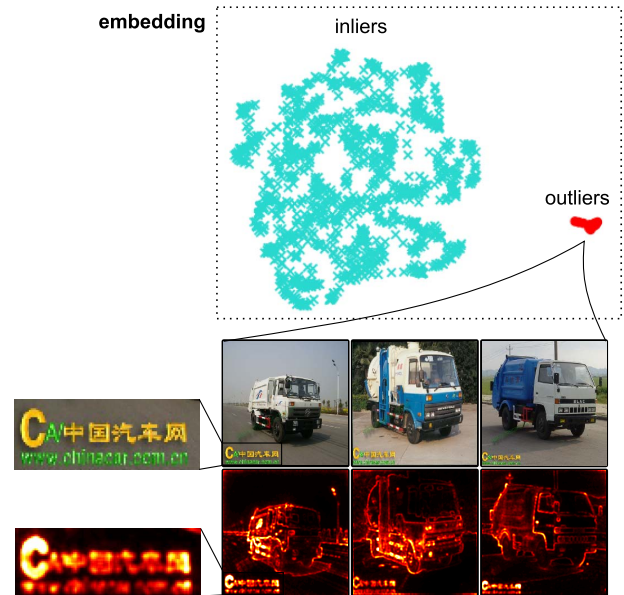
the goal of the analysis is to systematically find data artifacts picked up by the model (e.g., copyright tags in horse images), whereas the second analysis aims to carefully investigate the learning process of a deep model, in particular, the emergence of novel prediction strategies during training.

The process of systematically extracting data artifacts was automated by a method called SpRAY [104], where, after computing LRP-type explanations on a whole data set (see Section VI-D), a cluster-based analysis was applied to the collection of produced explanations to extract *prototypical* decision strategies. The SpRAY analysis would, for example, reveal for some shallow Fisher vector model trained on the Pascal VOC 2007 data set that images of the label “horse” would be predicted as such using a finite number of prototypical decision strategies ranging from detecting the horse itself to detecting weakly related features, such as horse racing poles, or clear artifacts, such as copyright tags [101]. The analysis was later applied to the decisions of a state-of-the-art VGG-16 deep neural network classifier trained on ImageNet, and here again, interesting insight about the decision structure could be identified [7]. Certain predictions, e.g., for the class “garbage truck,” could be found by SpRAY to rely on some watermark in the bottom-left corner of the image (see Fig. 18). This watermark, which is only present in specific images, would, thus, be used by the model as a confounding factor (or artifact) to artificially improve prediction accuracy on this benchmark.<sup>7</sup>

Such strategy employed by the ML classifier can be referred to as “Clever Hans” [104]. For ML models having implemented a Clever Hans strategy, an overconfident assessment of the true model accuracy would be produced by solely relying on the accuracy metric without an inspection of the model’s decision structure. The model would have likely performed erratically once it is applied in a real-world setting, where, e.g., the copyright tag is decoupled from the concept of a horse or garbage truck, respectively. Here, the ability to explain the decision-making of the model and to automatically analyze these explanations on a very large data set was, therefore, a key ingredient to more robustly assess the model’s strength and weakness and potentially improve it.

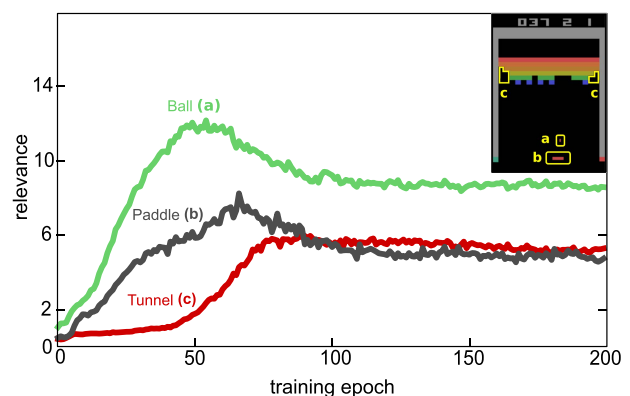
Another example of a systematic interpretation of ML models can be found in the context of reinforcement learning, in particular board and video games. Here, large amounts of data can be easily generated (simulated games) and used to carefully analyze the strategies of an ML model and how these strategies emerge during training. In games such as the arcade game Atari Breakout, the computer player would progressively learn strategies commonly employed by human players, such as “tunnel-digging” [124], [200]. The work of [104] analyzes the emergence of this advanced “tunnel-digging” technique

<sup>7</sup>Or in the case of [7] *deteriorate* model performance, as the identified confounding feature is exclusive to the training data.



**Fig. 18.** SpRAY analysis of the predictions of a pretrained VGG-16 model on images of the class “garbage truck.” Top: low-dimensional embedding of the explained decisions for the class “garbage truck.” Points highlighted in red are outliers. Bottom: images and corresponding decisions for some of the points highlighted in red.

using explanations. First, LRP-type pixelwise explanations of the player’s decision were produced at various time steps and training stages. The produced collection of explanations were then pooled (see Section VI-D1) on bounding boxes representing some key visual elements of the game, specifically the ball, the paddle, and the tunnel. Pooled quantities could then be easily and quantitatively monitored over the different stages of training. The analysis is shown in Fig. 19.



**Fig. 19.** Analysis of the learning process of a deep model playing Atari Breakout. The curves show the development of the relative relevance of different game objects (ball, paddle, and tunnel) averaged over six runs.



We observe that the neural network model would first learn to play conventionally by keeping track of the ball and the paddle and, only at a later stage of the training process, would learn to focus on the tunnel area, allowing the ball to go past the wall and bounce repeatedly in the top area of the screen. This analysis highlights in a way that is easily interpretable to the human the multistage nature of learning, in particular, how the learning machine progressively develops increasingly sophisticated game-playing strategies. Overall, this summarized information on the decision structure of the model and on the evolution of the learning process could prove to be crucial in learning improved models on purposely consolidated data sets. They could also prove useful for characterizing the different stages of learning and developing more efficient training procedures.

## B. From Explanations to Novel Scientific Insights

In Section VIII-A, we demonstrated the use of explanation techniques for systematically analyzing models and verifying that they have learned valid and meaningful prediction strategies. Once verified to not be Clever Hans predictors, nonlinear models offer a lot of potentials for the sciences to detect new interesting patterns in the data, which may lead to an improved understanding of the underlying natural structures and processes—the primary goal of scientists. So far, this was not possible because nonlinear models were actually considered to be “black boxes,” i.e., scientists had to resort to the use of linear models (see [63] and [117]), even if this came at the expense of predictivity. Only recently, the technologies have become available to extract scientific insights from complex nonlinear models [121], [150], [166]. In the following, we will present a selection of problems where ML explanation techniques bring the full potential of nonlinear methods to scientific disciplines.

Let us start with the discussion of scientific problems that concern images. These problems could directly benefit from the advances made in general image recognition in the last years.

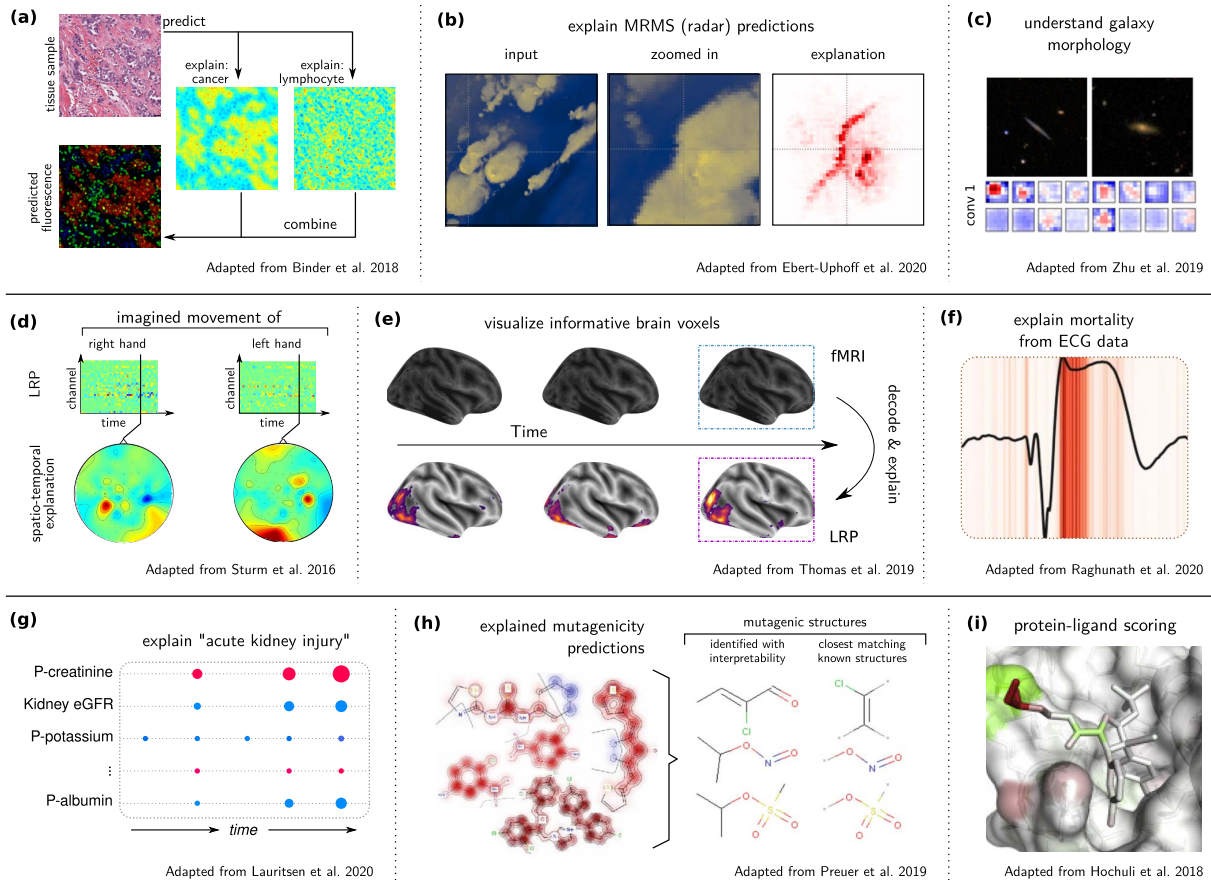
An important application area is in medicine [74]. Fig. 20(a) shows one such application: the task of predicting tissue type from histopathology imagery. The work of [22] proposes a bag-of-words model for the prediction task with invariances to the rotation, shift, and scale of the input data. For the verification of the prediction results, the LRP technique is applied to this model so that heatmaps are produced, offering per-pixel scores that indicate the presence of tumorous structures. Furthermore, LRP heatmaps computed for different target cell types can be combined for obtaining computationally predicted fluorescence images. The explanations are histopathologically meaningful and may potentially give interesting information about which tissue components are most indicative of cancer. Further analyses, such as the identification, localization, and counting of cells, i.e., lymphocytes, can

be performed on these explanations (see [93]). Recently, deeper models have also been used for predicting and explaining tissue type [61]. In addition to visual explanations, Zhang *et al.* [204] also generate a free-text pathology report to clarify the decision of the classifier. Further applications of DNN explanation techniques for medical images include the detection of lesions in diabetic retinopathy data [145], pixelwise analysis of microscopy images from global image annotations [98], the validation of predictions in dermatology [199], and analysis of X-ray images [48]. The latter work produces, in addition to the image-level explanations, descriptive sentences to further improve the interpretability of the ML decision for a doctor.

Methods for explaining DNNs have also been applied outside of medical imaging, for example, in earth sciences. Ebert-Uphoff and Hilburn [43] and Lee *et al.* [109] predict various meteorological values, such as convection, or MRMS radar, from satellite imagery, and the latter prediction is then attributed to the individual pixels. An example is shown in Fig. 20(b), where the MRMS output is supported by two visual patterns, the edge of the cloud, and a brighter spot representing a denser region of the cloud. Fig. 20(c) shows an ML-based analysis of galaxy morphologies [207]. The latter work aims to classify galaxy morphologies into five classes (completely round smooth, in-between smooth, cigar-shaped smooth, edge-on, and spiral) using a convolutional neural network, and the convolution filters and activation patterns are analyzed to understand what visual features are associated with these different classes [see Fig. 20(f)]. Finally, Ghosal *et al.* [50] also rely on training and explaining DNN image classifiers to get novel insights, this time in the area of plant stress classification.

Explanation methods for deep neural networks have also demonstrated their potential beyond the image domain, e.g., on scientific problems concerning time-series data. For instance, the work of [180] presents one of the first uses of DNNs explanation techniques in cognitive neurosciences, specifically in brain-computer interfacing [40] where linear methods are still the most widely used filtering methods [24], [63]. The results in [180] show that deep models achieve similar decoding performances<sup>8</sup> and learn neurophysiologically plausible patterns [see Fig. 20(b)], namely, focus on the contralateral sensorimotor cortex—an area where the event-related desynchronization occurs during motor imagery. However, in contrast to the patterns computed with conventional approaches [24], [63], which only allows visualizing the aggregated information (average activity) per class, the explanations computed with LRP are available for every single input of the deep learning classifier, i.e., for every time point of individual trials [see Fig. 20(d)]. This increased resolution (knowing which sources are relevant at each time point, instead of just having the

<sup>8</sup>Deep models usually require larger amounts of training data to have an advantage over linear techniques.



**Fig. 20.** Different applications of explanation techniques in the sciences. (a) LRP heatmaps merged into a computationally predicted fluorescence image. Here, red identifies cancer, green shows lymphocytes, and blue is a stroma. Adapted from [22]. (b) Prediction of MRMS radar signal from satellite image and pixelwise explanation. Adapted from [43]. (c) Learned filter weights from the first convolutional layer of a deep neural network trained for galaxy morphology classification. Adapted from [207]. (d) Whole-brain fMRI volume is decoded using a DNN. The decoding decision is explained voxelwise to localize brain areas corresponding to the predicted cognitive state. Adapted from [186]. (e) Example of LRP relevance maps for a single EEG trial of an imagined movement (each class). The matrices indicate the relevance of each time point (abscissa) and EEG channel (ordinate). Below the matrix, the relevance information for two single time points is plotted as a scalp topography. Adapted from [180]. (f) Prediction of mortality from ECG time-series data. Red areas indicate the time steps that most strongly explain the prediction. Adapted from [146]. (g) Some medical conditions predicted from electronic health record (EHR) time series and explained in terms of input features. Blue/red indicates low/high value, and the circle size indicates feature relevance. Adapted from [106]. (h) Graph convolutional neural prediction of the molecule's mutagenicity and attribution of individual atoms. Interpretability feedback reveals that the model has correctly identified molecular substructures known to interact with (human) DNA. Adapted from [144]. (i) Predicted atom score describing protein-ligand interaction is explained with CLRP (green corresponding to a more favorable score). Adapted from [71].

average patterns) sheds further light on cognitive processes in the brain.

Another application of DNN explanation techniques in cognitive neuroscience is presented in [186], where a deep model is learned to predict whole-brain fMRI data. The method, termed DeepLight, outperforms well-established local or linear decoding methods, such as the generalized linear model and searchlight (see [186]). An adaptation of LRP maintains interpretability and substantiates that the model's predictions are based on physiologically appropriate brain areas for the classified cognitive states. The approach is depicted in Fig. 20(e), which visualizes the exemplar voxels that are used by the deep model to accurately decode the state from the fMRI signal. These voxels of high relevance have been shown to correspond very

well to the active areas described in the fMRI literature (see [186]). Note that also here the deep model not only gives an advantage in terms of performance (i.e., better decoding accuracy) compared to the local or linear baseline methods but also its explanations are provided for every single input, i.e., for every fMRI volume over time. This increased resolution allows studying the spatiotemporal dynamics of the fMRI signal and its impact on decoding, something which is not possible with classical decoding methods.<sup>9</sup>

Many other studies use explanation methods to analyze time-series signals in the sciences. For example,

<sup>9</sup>In classical fMRI analyses, p-values indicate the relevance of brain voxels. However, these p-values are usually obtained on a subject or group-level, not for single trials or single time points.

Raghunath *et al.* [146] build a model of mortality from ECG data and apply DNN explanation techniques to identify individual subjects' patterns in the ECG time series that supports the prediction (see Fig. 20(f) for one such explanation). In another work [106], the input time series is formed by EHRs sampled at a much lower rate, from which a DNN model can decide among a set of medical conditions. The medical condition can then be attributed to the different medical measurements. An example of explanation is given in Fig. 20(g). The produced explanation provides valuable additional information to decide in favor or against a particular treatment. Horst *et al.* [77] introduce explanation techniques to the domain of human gait recognition and show that nonlinear learning models are not only the better predictors but also they can, at the same time, learn physiologically meaningful features for subject prediction, which aligns with expected features used by linear models. Another work [97] applies long short-term memory (LSTM) networks to the field of hydrology to predict the river discharge from meteorological observations. The authors apply the integrated gradients technique to analyze the internals of the network and obtain insights that are consistent with our understanding of the hydrological system.

Structured data, such as molecules or gene sequences, are another very important domain for scientific investigations. Therefore, nonlinear ML coupled with explanation techniques has also attracted attention in scientific communities working with this type of data. One successful example of the use of DNN explanation methods in this domain has been reported in [144]. The authors train a deep model to predict molecular properties and bioactivities and apply the integrated gradients method to analyze what the model has learned and extract interesting insights [see Fig. 20(h)]. For instance, they find that single neurons play the role of pharmacophore detectors and demonstrate that the model uses pharmacophore-like features to reach its conclusions, which are consistent with pharmacologist literature. Another work [71] [see Fig. 20(i)] applies an extended version of LRP called CLRP to visualize how CNNs interpret individual protein-ligand complexes in molecular modeling. Also, here, the trained model learns meaningful features and has the ability to provide new insights into the mechanisms underlying protein-ligand interactions. Yet, another work [196] applies LSTM predictors together with LRP for transparent therapy prediction on patients suffering from metastatic breast cancer. Clinical experts verify that the features used for prediction, as revealed via LRP, largely agree with established clinical guidelines and knowledge. The work by Kelley *et al.* [87] uses a gradient-based explanation technique to understand the activity prediction across chromosomes, whereas Chong *et al.* [31] use the LRP explanation technique for understanding automated decisions on behavioral biometrics. Recently, the physics community started to use ML explanations for the task of energy prediction. The work of [165] and [166] showed that accurate predictions are

possible and obtained also physical meaningful insights from the model. Other works [114] showed that explanations in the gene analysis lead to interpretable patterns consistent with literature knowledge.

## IX. CHALLENGES AND OUTLOOK

While recent years have seen astonishing conceptual and technical progress in XAI, it is important to carefully discuss the current limits and the challenges that will need to be addressed by researchers to further establish the field and increase the usefulness of XAI systems.

Foundational theoretical work in XAI has so far been limited. As discussed in Section V, Shapley values [116], [169] and (deep) Taylor decomposition [127] have been proposed as principled frameworks for formalizing the task of explanation. Other frameworks, such as the rate distortion theory [118], have also been proposed. Numerous theoretical questions, however, remain: For example, it remains unclear how to weigh the model and the data distribution into the explanation, in particular, whether an explanation should be based on *any* features the model locally reacts to, or only those that are expressed locally. Related to this question is that of causality, i.e., assuming a causal link between two input variables, it has not been answered yet whether the two variables, or only the source variable, must constitute the explanation. A deeper formalization and theoretical understanding of XAI will be instrumental for shedding light on these important questions.

Another central question in XAI is that of optimality of an explanation. So far, there is no well-agreed understanding of what should be an optimal explanation. Also, ground-truth explanations cannot be collected by humans as this would presuppose that they are able to make sense of the complex ML model that they would like to explain in the first place. Methods such as “pixel-flipping” [158] assess explanation quality indirectly by testing how flipping relevant pixels affects the output score. The “axiomatic approach” [125], [183] does not have this indirect step; however, axioms are usually too generic to evaluate an explanation comprehensively. Approaches relying on ground-truth information (e.g., [12]) derived from a synthetic data set offer a direct and objective way to evaluate and compare explanations; however, it needs to be demonstrated that the synthetic problem is representative of the typically more complex real-world problem, where the ground-truth explanations are usually not available. Self-explainable models (e.g., [154]) provide another potential avenue toward optimal explanations, by forcing the model to be built in a way that explanations can be unambiguously extracted, thereby bypassing some of the technical challenges of *post hoc* explanations. Such a self-explainable approach comes, however, at the possible expense of prediction accuracy or runtime efficiency. Hence, the question of producing optimal explanations is still largely an open question. Finally, accuracy is only one factor in a more

general assessment of the overall practical value of an explanation, which further includes human factors, such as understandability, manageability, and overall utility of the XAI system [133], [147], [184]. Application-driven evaluations account for these additional factors; however, they are also hard to implement in practice [41].

Further challenges arise when applying XAI on problems where different actors (e.g., the explainer and the explainee) have conflicting interests. Recent work has shown that an “adversary” can modify the ML model in an imperceptible fashion so that the prediction behavior remains intact, but the explanation of those predictions changes drastically [68]. Relatedly, even when the model remains unchanged, inputs could be perturbed imperceptibly to produce arbitrary explanations [39]. An important challenge will be to provide provable guarantees on the robustness of explanations to various types of external distortion.

Interpretability may also find itself at odds with the constant quest for higher predicting accuracy. The model development and the explanation of an already trained model are often treated as two independent processes, i.e., the model developers aim to build the best possible model, and the *post hoc* XAI community provides the tools to interpret it. Because highly predictive models are becoming increasingly complex both in terms of the number of parameters and structure of the model, XAI software must keep up with this increasing complexity [3], and at the same time, the human must also deal with explanations of increasingly subtler predictions [14]. The same challenges also occur for self-explainable models (see Section III-E), where the incorporated interpretability structures would have to be continuously refined and extended to cope with the increased complexity of data and tasks, and the intrinsic limits of the human receiving the explanations.

When designing new XAI-driven applications, adopting a holistic view that sets the right tradeoffs and delivers the optimal amount of information and range of action to the multiple and potentially conflicting actors, will constitute an important practical challenge.

Another question of utmost importance, especially, in safety-critical domains, is whether we can fully trust the model after having explained some predictions. Here, we need to distinguish between model interpretation and model certification: while it is helpful to explain models for available input data, e.g., to potentially detect some erroneous decision strategies, certification would require verifying the model for *all possible* inputs, not only those included in the data. Furthermore, it must be remembered that explanations returned to the user are summaries of a potentially complex decision process, i.e., there may be different decision strategies, the wrong ones and the correct ones, mapping to the same explanation. Finally, explanations are subject to their own biases and approximations, and they can be manipulated by an adversary to lose their informative content. Therefore, in order to

ultimately establish a truly safe and trustworthy model, further steps are needed, potentially including the use of formal verification methods [21], [84].

Moreover, it may be worthwhile to explore new forms of explanations that are optimally suited to their user. Such explanations could, for example, leverage the user’s prior knowledge or personal preferences. Novel approaches from knowledge engineering, cognitive sciences, and human–computer interfaces will need to contribute. Also, while heatmaps provide the first intuition to users, they may not take advantage of the complex abstract reasoning capabilities of humans and can be very difficult to interpret for certain data modalities (e.g., time-series data). As we have discussed in the second worked-through example, explaining using a different modality (or by projecting the data on a more interpretable basis) can lead to an explanation that is more useful to the user. An important challenge will be to develop systematic ways of transferring explanations from the original input domain to a more interpretable target domain.

Finally, we see as a key future challenge the design of XAI techniques that can automatically extract meaningful collective variables and explain their terms. Collective variables are central in many areas of physics. In solid-state physics, they have led to groundbreaking advances, defining quasiparticles, such as phonons, plasmons, polarons, magnons, and excitons [92]. Ideally, collective variables, in this sense, would be inferred from a learning model by, e.g., automatically binding explanation variables in meaningful abstract ways. In the neurosciences, von der Malsburg [192] has coined the concept of “binding” for neural strategies that allow sets of variables (neurons) to synchronize collectively by learning. While steps have already been taken in XAI to identify interacting variables [34], [42], [115], [162], [188], it will be necessary for a broader usage to generalize the approach, e.g., to extract “mathematical formulas” that first build the needed collective variables and use them to explain, concisely but deeply, the ML predictions.

## X. CONCLUSION

Complex nonlinear ML models, such as neural networks or kernel machines, have become game-changers in the sciences and industry. Fast progress in the field of XAI has made virtually any of these complex models, supervised or unsupervised, interpretable to the user. Consequently, we no longer need to give up predictivity in favor of interpretability, and we can take full advantage of strong nonlinear ML in practical applications.

In this review, we have made the attempt to provide a systematic path to bring XAI to the attention of an interested readership. This included an introduction to the technical foundations of XAI, a presentation of practical algorithms, such as occlusion, integrated gradients, and LRP, concrete examples illustrating how to use explanation techniques in practice and a discussion of successful applications. We would like to stress that the techniques intro-



duced in this article can be readily and broadly applied to the workhorses of supervised and unsupervised learnings, e.g., clustering, anomaly detection, kernel machines, deep networks, and state-of-the-art pretrained convolutional networks and LSTMs.

XAI techniques not only shed light on the inner workings of nonlinear learning machines, explaining why they arrive at their successful predictions; they also help to discover biases and quality issues in large data corpora with millions of examples [7]. This is an increasingly relevant direction since modern ML relies more and more on reference data sets and reference pretrained models. Furthermore, initial steps have been taken to use XAI beyond validation to arrive at better and more predictive models, e.g., [6]–[8] and [152].

We would like to stress the importance of XAI, notably in safety-critical operations, such as medical assistance or diagnosis, where the highest level of transparency is required in order to avoid fatal outcomes.

Finally, as a versatile tool in the sciences, XAI has been allowing to gain novel insights (e.g., [22], [45], [71], [140], [159], [165], [186]) ultimately contributing to further our scientific knowledge.

While XAI has seen an almost exponential rise in interest (and progress) with communities forming and many workshops emerging, there is a wealth of open problems and challenges with ample opportunities to contribute (see Section IX). Concluding, we firmly believe that XAI will, in the future, become an indispensable practical ingredient to obtain improved, transparent, safe, fair, and unbiased learning models.

## APPENDIX A IMPLEMENTING SMOOTH INTEGRATED GRADIENTS

In this appendix, we present the algorithm combining SmoothGrad [176] and integrated gradients [183], which we use in Section IV in our comparison of explanation methods. Its implementation is shown in Algorithm 5.

### Algorithm 5 Integrated Gradients With Smoothing

```

R = 0
for  $s = 1 \dots S$  do
   $\tilde{x} \sim \mathcal{N}(0, \sigma I)$ 
  for  $t = 1 \dots T$  do
     $R = R + (x - \tilde{x}) \odot \nabla f(\tilde{x} + \frac{t-0.5}{T} \cdot (x - \tilde{x}))$ 
  end for
end for
return  $\frac{1}{TS} \cdot R$ 

```

The procedure consists of a simple nested loop of  $S$  smoothing and  $T$  integration steps, where each integration starts at some random location near the origin. Here, we note that these locations are not strict root points. However, in the context of image data, random noise does not significantly change evidence in favor or against a

particular class. Thus, the explanation remains approximately complete.

## APPENDIX B IMPLEMENTING LAYERWISE RELEVANCE PROPAGATION

In this appendix, we outline two possible implementations of LRP [13], [126]. The first one is intuitive and based on looping forward and backward over the multiple layers of the neural network. This procedure can be applied to simple sequential structures, such as VGG-16 [175]. The second approach that we present is based on “forward hooks” and serves to extend the LRP method to more complex architectures, such as ResNet [65].

### A. Standard Sequential Implementation

The standard implementation is based on the forward-backward procedure outlined in Algorithm 2. We focus here on the `relprop` function of this procedure, which is called at each layer to propagate relevance to the layer below. We give an implementation for the LRP-0/ $\epsilon/\gamma$  rules [13], [126] and one for the  $z^B$ -rule [127]. The first three rules can be seen as special cases of the more general rule

$$R_j = \sum_k \frac{a_j \rho(w_{jk})}{\epsilon + \sum_{0,j} a_j \rho(w_{jk})} R_k$$

where  $\rho(w_{jk}) = w_{jk} + \gamma w_{jk}^+$ . This propagation rule can be computed in four steps.

### Algorithm 6 LRP-0/ $\epsilon/\gamma$

```

 $z = \epsilon + f_l^p(a^{(l-1)})$  (Step 1)
 $s = R^{(l)} \oslash z$  (Step 2)
 $c = \nabla \langle z, [s]_{\text{cst.}} \rangle$  (Step 3)
 $R^{(l-1)} = a^{(l-1)} \odot c$  (Step 4)
return  $R^{(l-1)}$ 

```

The first step applies  $f_l^p$ , a forward evaluation of a copy of the layer whose parameters have gone through some function  $\rho$ , and also adds a small positive term  $\epsilon$ . The second step applies an elementwise division (denoted by “ $\oslash$ ”). The third step is conveniently expressed as a gradient of some dot product  $\langle z, [s]_{\text{cst.}} \rangle$  with respect to the input activations. The notation  $[\cdot]_{\text{cst.}}$  indicates that the term has been detached from the gradient computation and is, therefore, treated as a constant. In PyTorch, for example, this can be achieved by calling `() .data`. The `relprop` function implemented by Algorithm 6 is applicable for most linear and convolution layers of a deep rectifier network. For the pixel layer, we use instead the  $z^B$ -rule [126], [127]

$$R_i = \sum_j \frac{x_i w_{ij} - l_i w_{ij}^+ - h_i w_{ij}^-}{\sum_i x_i w_{ij} - l_i w_{ij}^+ - h_i w_{ij}^-} R_j$$

where  $l_i$  and  $h_i$  are the lowest/highest possible pixel values of  $x_i$ , and  $w_{ij}^- = \min(0, w_{ij})$ . The corresponding implementation is shown in Algorithm 7 and again consists of four steps.

---

**Algorithm 7**  $z^B$ -Rule

---

```

 $z = f_1(x) - f_1^+(l) - f_1^-(h)$  (Step 1)
 $s = R^{(1)} \odot z$  (Step 2)
 $c = \nabla_{x,l,h} \langle z, [s]_{\text{cst.}} \rangle$  (Step 3)
 $R^{(0)} = x \odot c_1 + l \odot c_2 + h \odot c_3$  (Step 4)
return  $R^{(0)}$ 

```

---

The functions  $f_1^+$  and  $f_1^-$  are forward passes on copies of the first layer whose parameters have been processed by the functions  $\max(0, \cdot)$  and  $\min(0, \cdot)$ , respectively.

## B. Forward-Hook Implementation

When the architecture has nonsequential components (e.g., ResNet [65]), it is more convenient to reuse the graph traversing procedures readily implemented by the model's existing forward pass and the automatically generated gradient propagation pass. To achieve this, we can implement “forward hooks” at each linear and convolution layer. In this case, we leverage the “smooth gradient” view of LRP [see (3)] and modify the implementation of the forward pass in a way that it keeps the forward pass functionally equivalent but modifies the local gradient computation. This is achieved by strategically detaching terms from the gradient in a way that calling the gradient becomes equivalent to computing (3) at each layer. Once the forward functions have been redefined at each layer, the explanation can be computed globally by calling the gradient of the whole function, as shown in Algorithm 8. (Note that, unlike the original function  $f(x)$ , the new function that includes the hooks receives three arguments as input: the data point  $x$  and the bounds  $l$  and  $h$  used by the first layer.)

---

**Algorithm 8** LRP Implementation Based on Forward Hooks

---

*Forward hook for intermediate layers (LRP-0/ $\epsilon/\gamma$ )*

```

 $z = \epsilon + f_l^p(a^{(l-1)})$ 
return  $z \odot [f_l(a^{(l-1)}) \odot z]_{\text{cst.}}$ 

```

---

*Forward hook for the first layer ( $z^B$ -rule)*

```

 $z = f_1(x) - f_1^+(l) - f_1^-(h)$ 
return  $z \odot [f_1(x) \odot z]_{\text{cst.}}$ 

```

---

*Global LRP computation*

```

 $y = f(x, l, h)$ 
 $c_1, c_2, c_3 = \hat{\nabla} y$ 
 $R = x \odot c_1 + l \odot c_2 + h \odot c_3$ 
return  $R$ 

```

---

The forward-hook implementation produces exactly the same output as the original function  $f(x)$ , but its “gradient,” which we denote by  $\hat{\nabla}$ , is no longer the same due to the detached terms. As a result, calling the gradient of this function and recombining it with the input yield the same desired LRP explanation as one would get with the standard LRP implementation but has now gained applicability to a broader set of neural network architectures.

## APPENDIX C EXPLANATION SOFTWARE

The attention to interpretability in ML has grown frantically throughout the past decade alongside research on and the development of computationally efficient deep learning frameworks. This attention, in turn, caused a strong demand for accessible and efficient software solutions for the out-of-the-box applicability of XAI. In this section, we briefly highlight a collection of software toolboxes released in recent years, providing convenient access to a plethora of methods of XAI and supporting various

**Table 2** Interpretability Software Packages by Time of Release

| Software Package  | Release     | Available from <a href="https://github.com/">https://github.com/</a> ...                           | Compute Backend               | GPU Support                          | Methods                                                                                     |
|-------------------|-------------|----------------------------------------------------------------------------------------------------|-------------------------------|--------------------------------------|---------------------------------------------------------------------------------------------|
| LRP Toolbox [102] | 2016        | <a href="https://github.com/sebastian-lapuschkin/lrp_toolbox">sebastian-lapuschkin/lrp_toolbox</a> | Caffe<br>numpy/cupy<br>Matlab | $\times$<br>$\checkmark$<br>$\times$ | DCN, DTD, GB, LRP, SA<br>LRP, SA<br>LRP, SA                                                 |
| DeepExplain [4]   | 2017        | <a href="https://github.com/marcoancona/DeepExplain">marcoancona/DeepExplain</a>                   | Keras+TensorFlow              | $\checkmark$                         | DLR, IG, LRP- $\epsilon$<br>SA, AS, OCC                                                     |
| iNNvestigate [3]  | 2019        | <a href="https://github.com/albermax/innvestigate">albermax/innvestigate</a>                       | Keras+TensorFlow              | $\checkmark$                         | DCN, DL, DTD, GB,<br>IG, LRP, PA, PN, SA, SG<br>Perturbation Analysis                       |
| TorchRay [46]     | 2019        | <a href="https://github.com/facebookresearch/TorchRay">facebookresearch/TorchRay</a>               | PyTorch                       | $\checkmark$                         | DCN, EB, EP<br>GC, GB, RISE, SA,<br>various benchmarks                                      |
| Captum [95]       | 2019 (beta) | <a href="https://github.com/pytorch/captum">pytorch/captum</a>                                     | PyTorch                       | $\checkmark$                         | DCN, DLR, DLShap,<br>GB, GC, GGC, GSHAP, IG, II, LC,<br>NC, NGB, NIG, SA, SG, SG-SQ, TC, VG |

**Table 3** Glossary of Interpretability Methods With Abbreviations Referenced Throughout Our Review

| Method                                    | Abbrev. | Method                                          | Abbrev.      |
|-------------------------------------------|---------|-------------------------------------------------|--------------|
| Anchors                                   | [148]   | ANCH                                            |              |
| ApproShapley (Shapley Value Sampling)     | [29]    | AS                                              |              |
| Class Activation Mapping                  | [206]   | CAM                                             |              |
| Contextual Prediction Difference Analysis | [56]    | CPDA                                            |              |
| DeconvNet                                 | [201]   | DCN                                             |              |
| DeepLIFT                                  | [170]   | DL                                              |              |
| DeepLIFT (Rescale)                        | [170]   | DLR                                             |              |
| DeepLIFT SHAP                             | [116]   | DLSHAP                                          |              |
| Deep Taylor Decomposition                 | [127]   | DTD                                             |              |
| ExcitationBackprop                        | [202]   | EB                                              |              |
| ExtremalPerturbation                      | [46]    | EP                                              |              |
| GNNExplainer                              | [198]   | GNNEXP                                          |              |
| GNN-LRP                                   | [162]   | GLRP                                            |              |
| GradCAM                                   | [167]   | GC                                              |              |
| Gradient SHAP                             | [116]   | GSHAP                                           |              |
| Gradient $\times$ Input                   | [170]   | GI                                              |              |
| GuidedBackprop                            | [178]   | GB                                              |              |
| Guided GradCam                            | [167]   | GGC                                             |              |
| Integrated Gradients                      | [183]   | IG                                              |              |
| Internal Influence                        | [110]   | II                                              |              |
| Kernel SHAP                               | [116]   | KSHAP                                           |              |
| LayerConductance                          | [172]   | LC                                              |              |
| Local Rule-based Explanations             | [58]    | LORE                                            |              |
|                                           |         | Layer-wise Relevance Propagation (full)         | [13]         |
|                                           |         | LRP (composite strategy)                        | [103], [126] |
|                                           |         | LRP (specific variants)                         | [13], [126]  |
|                                           |         | Local Interpretable Model-agnostic Explanations | [147]        |
|                                           |         | Meaningful Perturbation                         | [47]         |
|                                           |         | NeuronConductance                               | [36]         |
|                                           |         | NeuronGuidedBackprop                            | [178]        |
|                                           |         | NeuronIntegratedGradients                       | [172]        |
|                                           |         | Occlusion Analysis                              | [201]        |
|                                           |         | PatternAttribution                              | [90]         |
|                                           |         | PatternNet                                      | [90]         |
|                                           |         | Prediction Difference Analysis                  | [208]        |
|                                           |         | Randomized Input Sampling for Explanation       | [142]        |
|                                           |         | Saliency Analysis / Gradient                    | [14], [174]  |
|                                           |         | SHapley Additive exPlanations                   | [116]        |
|                                           |         | SHAP Interaction Index                          | [115]        |
|                                           |         | SmoothGrad                                      | [176]        |
|                                           |         | SmoothGrad <sup>2</sup>                         | [76]         |
|                                           |         | Spectral Relevance Analysis                     | [104]        |
|                                           |         | TreeExplainer                                   | [115]        |
|                                           |         | VarGrad                                         | [1]          |
|                                           |         | Testing with Concept Activation Vectors         | [89]         |
|                                           |         | TotalConductance                                | [36]         |
|                                           |         | LRP-CMP                                         |              |
|                                           |         | LRP-*                                           |              |
|                                           |         | LIME                                            |              |
|                                           |         | MP                                              |              |
|                                           |         | NC                                              |              |
|                                           |         | NGB                                             |              |
|                                           |         | NIG                                             |              |
|                                           |         | OCC                                             |              |
|                                           |         | PA                                              |              |
|                                           |         | PN                                              |              |
|                                           |         | PDA                                             |              |
|                                           |         | RISE                                            |              |
|                                           |         | SA                                              |              |
|                                           |         | SHAP                                            |              |
|                                           |         | SHAPIDX                                         |              |
|                                           |         | SG                                              |              |
|                                           |         | SG-SQ                                           |              |
|                                           |         | SpRAY                                           |              |
|                                           |         | TEXP                                            |              |
|                                           |         | VG                                              |              |
|                                           |         | TCAV                                            |              |
|                                           |         | TC                                              |              |

computational back ends. A summarizing overview of the presented software solutions is given in Table 2, alongside a glossary of methods with respective abbreviations used throughout our review in Table 3.

One of the earlier and comprehensive XAI software packages is the LRP Toolbox [102], providing presently up to date implementations of LRP for the—until very recently—popular Caffe deep learning framework [83], as well as MATLAB and Python via custom neural network interfaces. While the support for Caffe is restricted to the C++ programming language and, thus, CPU hardware, it provides functionality implementing DCN, GB, DTD, and SA and can be built and used as a stand-alone executable binary for predictors based on the Caffe neural network format. The subpackages available for MATLAB and Python provide out-of-the-box support for LRP and SA while being easily extensible via custom neural network modules written with clarity and the methods' intelligibility in mind. The *cupy* [137] back end constitutes an alternative to the CPU-bound *numpy* [138] package, providing optional support for modern GPU hardware from NVIDIA.

Both the DeepExplain [4] and iNNvestigate [3] toolboxes are built on top of the popular Keras [30] package for Python with TensorFlow back end for explaining deep neural network models and, thus, provide

support for both CPU and GPU hardware and convenient access for users of Keras models. While the more recent iNNvestigate Toolbox implements a superset of the modified backpropagation methods available in DeepExplain, the latter also offers functionality for perturbation-based attribution methods, i.e., the occlusion method [201] and Shapley value resampling [29]. For explaining a model's prediction, DeepExplain allows for an *ad hoc* selection of the explanation method via pythonic context managers. The iNNvestigate package on the other hand operates by attaching and automatically configuring (several) modified backward graphs called “analyzers” to a model of interest—one per XAI method to compute attributions with.

A present trend in the ML community is a migration to the PyTorch framework with its eager execution paradigm, away from other back ends. Both the TorchRay [46] and Captum [95] packages for Python and PyTorch enable the use of interpretability methods for neural network models defined in the context of PyTorch's high-level neural network description modules. Captum can be understood as a rich selection of XAI methods based on modified backprop and is part of the PyTorch project itself. While not as extensive as Captum, the TorchRay package offers a series of benchmarks for XAI alongside its selection of (benchmarked) interpretability methods. ■

## REFERENCES

- [1] J. Adebayo, J. Gilmer, I. J. Goodfellow, and B. Kim, “Local explanation methods for deep neural networks lack sensitivity to parameter values,” in *Proc. Int. Conf. Learn. Represent.*, 2018.
- [2] C. Agarwal and A. Nguyen, “Explaining image classifiers by removing input features using generative models,” in *Proc. Asian Conf. Comput. Vis.*, 2020.
- [3] M. Alber et al., “iNNvestigate neural networks,” *J. Mach. Learn. Res.*, vol. 20, no. 93, pp. 93:1–93:8, 2019.
- [4] M. Ancona, E. Ceolini, C. Öztireli, and M. Gross, “Towards better understanding of gradient-based attribution methods for deep neural networks,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2018.
- [5] M. Ancona, C. Öztireli, and M. Gross, “Explaining deep neural networks with a polynomial time algorithm for Shapley value approximation,” in *Proc. 36th Int. Conf. Mach. Learn.*, vol. 97, 2019, pp. 272–281.
- [6] C. J. Anders, G. Montavon, W. Samek, and K.-R. Müller, “Understanding patch-based learning of video data by explaining predictions,” in *Explainable AI: Interpreting, Explaining and Visualizing Deep Learning*. Cham, Switzerland: Springer, 2019, pp. 297–309.
- [7] C. J. Anders, L. Weber, D. Neumann, W. Samek,

- K.-R. Müller, and S. Lapuschkin, "Finding and removing clever hans: Using explanation methods to debug and improve deep models," 2019, *arXiv:1912.11425*. [Online]. Available: <http://arxiv.org/abs/1912.11425>
- [8] J. A. Arjona-Medina, M. Gillhofer, M. Widrich, T. Unterthiner, J. Brandstetter, and S. Hochreiter, "Rudder: Return decomposition for delayed rewards," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 13544–13555.
- [9] L. Arras et al., "Explaining and interpreting LSTMs," in *Explainable AI: Interpreting, Explaining and Visualizing Deep Learning* (Lecture Notes in Computer Science), vol. 11700. Cham, Switzerland: Springer, 2019, pp. 211–238.
- [10] L. Arras, F. Horn, G. Montavon, K.-R. Müller, and W. Samek, "What is relevant in a text document?: An interpretable machine learning approach," *PLoS ONE*, vol. 12, no. 8, 2017, Art. no. e0181142.
- [11] L. Arras, G. Montavon, K.-R. Müller, and W. Samek, "Explaining recurrent neural network predictions in sentiment analysis," in *Proc. 8th Workshop Comput. Approaches to Subjectivity, Sentiment Social Media Anal.*, 2017, pp. 159–168.
- [12] L. Arras, A. Osman, and W. Samek, "Ground truth evaluation of neural network explanations with CLEVR-XAI," 2020, *arXiv:2003.07258*. [Online]. Available: <http://arxiv.org/abs/2003.07258>
- [13] S. Bach, A. Binder, G. Montavon, F. Klauschen, K.-R. Müller, and W. Samek, "On pixel-wise explanations for non-linear classifier decisions by layer-wise relevance propagation," *PLoS ONE*, vol. 10, no. 7, Jul. 2015, Art. no. e0130140.
- [14] D. Baehrens, T. Schroeter, S. Harmeling, M. Kawanabe, K. Hansen, and K.-R. Müller, "How to explain individual classification decisions," *J. Mach. Learn. Res.*, vol. 11, pp. 1803–1831, Aug. 2010.
- [15] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *Proc. 3rd Int. Conf. Learn. Represent.*, 2015.
- [16] D. Balduzzi, M. Frean, L. Leary, J. P. Lewis, K. W. Ma, and B. McWilliams, "The shattered gradients problem: If resnets are the answer, then what is the question," in *Proc. 34th Int. Conf. Mach. Learn.*, vol. 70, 2017, pp. 342–350.
- [17] D. Bau, J. Zhu, H. Strobelt, B. Zhou, J. B. Tenenbaum, W. T. Freeman, and A. Torralba, "GAN dissection: Visualizing and understanding generative adversarial networks," in *Proc. 7th Int. Conf. Learn. Represent.*, 2019.
- [18] D. Bau, J.-Y. Zhu, H. Strobelt, A. Lapedriza, B. Zhou, and A. Torralba, "Understanding the role of individual units in a deep neural network," *Proc. Nat. Acad. Sci. USA*, vol. 117, no. 48, pp. 30071–30078, Dec. 2020.
- [19] S. Bazen and X. Joutard, "The Taylor decomposition: A unified generalization of the Oaxaca method to nonlinear models," HAL, Aix-Marseille School Econ., Marseille, France, Working Papers, 2013.
- [20] S. Becker, M. Ackermann, S. Lapuschkin, K.-R. Müller, and W. Samek, "Interpreting and explaining deep neural networks for classification of audio signals," 2018, *arXiv:1807.03418*. [Online]. Available: <http://arxiv.org/abs/1807.03418>
- [21] F. Berkenkamp, M. Turchetta, A. Schoellig, and A. Krause, "Safe model-based reinforcement learning with stability guarantees," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 908–918.
- [22] A. Binder et al., "Morphological and molecular breast cancer profiling through explainable machine learning," *Nature Mach. Intell.*, 2021, doi: 10.1038/s42256-021-00303-4.
- [23] C. M. Bishop, *Neural Networks for Pattern Recognition*. New York, NY, USA: Oxford Univ. Press, 1996.
- [24] B. Blankertz, R. Tomioka, S. Lemm, M. Kawanabe, and K.-R. Müller, "Optimizing spatial filters for robust EEG single-trial analysis," *IEEE Signal Process. Mag.*, vol. 25, no. 1, pp. 41–56, Dec. 2008.
- [25] W. Brendel and M. Bethge, "Approximating CNNs with bag-of-local-features models works surprisingly well on imagenet," in *Proc. 7th Int. Conf. Learn. Represent.*, 2019.
- [26] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2017, pp. 39–57.
- [27] M. Caron, P. Bojanowski, A. Joulin, and M. Douze, "Deep clustering for unsupervised learning of visual features," in *Proc. 15th Eur. Conf. Comput. Vis.*, in Lecture Notes in Computer Science, vol. 11218. Cham, Switzerland: Springer, 2018, pp. 139–156.
- [28] R. Caruana, Y. Lou, J. Gehrke, P. Koch, M. Sturm, and N. Elhadad, "Intelligible models for healthcare: Predicting pneumonia risk and hospital 30-day readmission," in *Proc. 21th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2015, pp. 1721–1730.
- [29] J. Castro, D. Gómez, and J. Tejada, "Polynomial calculation of the Shapley value based on sampling," *Comput. Oper. Res.*, vol. 36, no. 5, pp. 1726–1730, May 2009.
- [30] F. Chollet et al. (2015). *Keras*. [Online]. Available: <https://keras.io>
- [31] P. Chong, Y. X. M. Tan, J. Guarnizo, Y. Elovici, and A. Binder, "Mouse authentication without the temporal aspect—What does a 2D-CNN learn?" in *Proc. IEEE Secur. Privacy Workshops (SPW)*, May 2018, pp. 15–21.
- [32] C. S. Cox, "Plan operation NHANES I epidemiologic followup study, 1987," U.S. Dept. Health Hum. Services, Public Health Service, Centers Disease Control, Nat. Center Health Statist., Hyattsville, MD, USA, Tech. Rep. 92-1303, 1992.
- [33] D. R. Cox, "Regression models and life-tables," *J. Roy. Stat. Soc. B, Methodol.*, vol. 34, no. 2, pp. 187–202, Jan. 1972.
- [34] T. Cui, P. Marttinen, and S. Kaski, "Learning global pairwise interactions with Bayesian neural networks," in *Proc. 24th Eur. Conf. Artif. Intell.*, vol. 325, 2020, pp. 1087–1094.
- [35] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2009, pp. 248–255.
- [36] K. Dhamdhere, M. Sundararajan, and Q. Yan, "How important is a neuron?" 2018, *arXiv:1805.12233*. [Online]. Available: <http://arxiv.org/abs/1805.12233>
- [37] I. S. Dhillon, Y. Guan, and B. Kulis, "Kernel k-means: Spectral clustering and normalized cuts," in *Proc. 10th ACM SIGKDD Int. Conf. Knowledge Discovery Data Mining*, 2004, pp. 551–556.
- [38] Y. Ding, Y. Liu, H. Luan, and M. Sun, "Visualizing and understanding neural machine translation," in *Proc. 55th Annu. Meeting Assoc. Comput. Linguistics*, 2017, pp. 1150–1159.
- [39] A. Dombrowski, M. Alber, C. J. Anders, M. Ackermann, K.-R. Müller, and P. Kessel, "Explanations can be manipulated and geometry is to blame," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 13567–13578.
- [40] G. Dornhege et al., *Toward Brain-Computer Interfacing*, vol. 63. Cambridge, MA, USA: MIT Press, 2007.
- [41] F. Doshi-Velez and B. Kim, "Towards a rigorous science of interpretable machine learning," 2017, *arXiv:1702.08608*. [Online]. Available: <http://arxiv.org/abs/1702.08608>
- [42] O. Eberle, J. Büttner, F. Kräutli, K.-R. Müller, M. Valleriani, and G. Montavon, "Building and interpreting deep similarity models," *IEEE Trans. Pattern Anal. Mach. Intell.*, to be published.
- [43] I. Ebert-Uphoff and K. Hilburn, "Evaluation, tuning, and interpretation of neural networks for working with images in meteorological applications," *Bull. Amer. Meteorological Soc.*, vol. 101, no. 12, pp. E2149–E2170, 2020.
- [44] E. Eidinger, R. Enbar, and T. Hassner, "Age and gender estimation of unfiltered faces," *IEEE Trans. Inf. Forensics Security*, vol. 9, no. 12, pp. 2170–2179, Dec. 2014.
- [45] H. J. Escalante, S. Escalera, I. Guyon, X. Baró, Y. Güçlütürk, U. Güçlü, and M. van Gerven, *Explainable and Interpretable Models in Computer Vision and Machine Learning*. Cham, Switzerland: Springer, 2018.
- [46] R. Fong, M. Patrick, and A. Vedaldi, "Understanding deep networks via extremal perturbations and smooth masks," in *Proc. IEEE Int. Conf. Comput. Vis.*, Oct. 2019, pp. 2950–2958.
- [47] R. C. Fong and A. Vedaldi, "Interpretable explanations of black boxes by meaningful perturbation," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 3449–3457.
- [48] W. Gale, L. Oakden-Rayner, G. Carneiro, L. J. Palmer, and A. P. Bradley, "Producing radiologist-quality reports for interpretable deep learning," in *Proc. 16th IEEE Int. Symp. Biomed. Imag.*, Apr. 2019, pp. 1275–1279.
- [49] J. Gama, I. Zliobaite, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Comput. Surv.*, vol. 46, no. 4, pp. 44:1–44:37, 2014.
- [50] S. Ghosal, D. Blystone, A. K. Singh, B. Ganapathysubramanian, A. Singh, and S. Sarkar, "An explainable deep machine vision framework for plant stress phenotyping," *Proc. Nat. Acad. Sci. USA*, vol. 115, no. 18, pp. 4613–4618, 2018.
- [51] X. Glorot, A. Bordes, and Y. Bengio, "Deep sparse rectifier neural networks," in *Proc. 14th Int. Conf. Artif. Intell. Statist.*, vol. 15, 2011, pp. 315–323.
- [52] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [53] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," in *Proc. 3rd Int. Conf. Learn. Represent.*, 2015.
- [54] B. Goodman and S. R. Flaxman, "European Union regulations on algorithmic decision-making and a 'right to explanation,'" *AI Mag.*, vol. 38, no. 3, pp. 50–57, 2017.
- [55] M. Grabisch and M. Roubens, "An axiomatic approach to the concept of interaction among players in cooperative games," *Int. J. Game Theory*, vol. 28, no. 4, pp. 547–565, Nov. 1999.
- [56] J. Gu and V. Tresp, "Contextual prediction difference analysis for explaining individual image classifications," 2019, *arXiv:1910.09086*. [Online]. Available: <http://arxiv.org/abs/1910.09086>
- [57] J. Gu, Y. Yang, and V. Tresp, "Understanding individual decisions of CNNs via contrastive backpropagation," in *Proc. 14th Asian Conf. Comput. Vis.*, in Lecture Notes in Computer Science, vol. 11363. Cham, Switzerland: Springer, 2018, pp. 119–134.
- [58] R. Guidotti, A. Monreale, S. Ruggieri, D. Pedreschi, F. Turini, and F. Giannotti, "Local rule-based explanations of black box decision systems," 2018, *arXiv:1805.10820*. [Online]. Available: <http://arxiv.org/abs/1805.10820>
- [59] R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, F. Giannotti, and D. Pedreschi, "A survey of methods for explaining black box models," *ACM Comput. Surv.*, vol. 51, no. 5, pp. 93:1–93:42, 2019.
- [60] A. Gupta, J. Johnson, L. Fei-Fei, S. Savarese, and A. Alahi, "Social GAN: Socially acceptable trajectories with generative adversarial networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 2255–2264.
- [61] M. Hägele et al., "Resolving challenges in deep learning-based analyses of histopathological images using explanation methods," *Sci. Rep.*, vol. 10, no. 1, p. 6423, Dec. 2020.
- [62] K. Hansen, D. Baehrens, T. Schroeter, M. Rupp, and K.-R. Müller, "Visual interpretation of kernel-based prediction models," *Mol. Informat.*, vol. 30, no. 9, pp. 817–826, Sep. 2011.



- [63] S. Haufe et al., "On the interpretation of weight vectors of linear models in multivariate neuroimaging," *NeuroImage*, vol. 87, pp. 96–110, Feb. 2014.
- [64] D. M. Hawkins, "The detection of errors in multivariate data using principal components," *J. Amer. Stat. Assoc.*, vol. 69, no. 346, pp. 340–344, Jun. 1974.
- [65] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
- [66] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T. Chua, "Neural collaborative filtering," in *Proc. 26th Int. Conf. World Wide Web*, 2017, pp. 173–182.
- [67] L. A. Hendricks, Z. Akata, M. Rohrbach, J. Donahue, B. Schiele, and T. Darrell, "Generating visual explanations," in *Proc. 14th Eur. Conf. Comput. Vis.*, in Lecture Notes in Computer Science, vol. 9908. Cham, Switzerland: Springer, 2016, pp. 3–19.
- [68] J. Heo, S. Joo, and T. Moon, "Fooling neural network interpretations via adversarial model manipulation," in *Proc. Adv. Neural Inf. Process. Syst.* 32, 2019, pp. 2921–2932.
- [69] M. A. Hernán and S. R. Cole, "Invited commentary: Causal diagrams and measurement bias," *Amer. J. Epidemiology*, vol. 170, no. 8, pp. 959–962, Oct. 2009.
- [70] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [71] J. Hochuli, A. Helbling, T. Skaist, M. Ragoza, and D. R. Koes, "Visualizing convolutional neural network protein-ligand scoring," *J. Mol. Graph. Model.*, vol. 84, pp. 96–108, Sep. 2018.
- [72] H. Hoffmann, "Kernel PCA for novelty detection," *Pattern Recognit.*, vol. 40, no. 3, pp. 863–874, Mar. 2007.
- [73] A. Holzinger, "From machine learning to explainable AI," in *Proc. World Symp. Digit. Intell. Syst. Mach. (DISA)*, Aug. 2018, pp. 55–66.
- [74] A. Holzinger, G. Langs, H. Denk, K. Zatloukal, and H. Müller, "Causability and explainability of artificial intelligence in medicine," *WIREs Data Mining Knowl. Discovery*, vol. 9, no. 4, p. e1312, Jul. 2019.
- [75] S. Hong, D. Yang, J. Choi, and H. Lee, "Interpretable text-to-image synthesis with hierarchical semantic layout generation," in *Explainable AI: Interpreting, Explaining and Visualizing Deep Learning* (Lecture Notes in Computer Science), vol. 11700. Cham, Switzerland: Springer, 2019, pp. 77–95.
- [76] S. Hooker, D. Erhan, P.-J. Kindermans, and B. Kim, "A benchmark for interpretability methods in deep neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 9734–9745.
- [77] F. Horst, S. Lapuschkin, W. Samek, K.-R. Müller, and W. I. Schöllhorn, "Explaining the unique nature of individual gait patterns with deep learning," *Sci. Rep.*, vol. 9, p. 2391, Feb. 2019.
- [78] G. B. Huang, M. Ramesh, T. Berg, and E. Learned-Miller, "Labeled faces in the wild: A database for studying face recognition in unconstrained environments," Dept. Comput. Sci., Univ. Massachusetts, Amherst, MA, USA, Tech. Rep. 07-49, 2007.
- [79] F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, "SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <1mb model size," 2016, *arXiv:1602.07360*. [Online]. Available: <http://arxiv.org/abs/1602.07360>
- [80] B. K. Iwana, R. Kuroki, and S. Uchida, "Explaining convolutional neural networks using softmax gradient layer-wise relevance propagation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. Workshop*, Oct. 2019, pp. 4176–4185.
- [81] J. D. Janizek, P. Sturmfels, and S.-I. Lee, "Explaining explanations: Axiomatic feature interactions for deep networks," 2020, *arXiv:2002.04138*. [Online]. Available: <http://arxiv.org/abs/2002.04138>
- [82] M. H. Jarrahi, "Artificial intelligence and the future of work: human-AI symbiosis in organizational decision making," *Bus. Horizons*, vol. 61, no. 4, pp. 577–586, Jul. 2018.
- [83] Y. Jia et al., "Caffe: Convolutional architecture for fast feature embedding," in *Proc. 22nd ACM Int. Conf. Multimedia*, 2014, pp. 675–678.
- [84] G. Katz, C. W. Barrett, D. L. Dill, K. Julian, and M. J. Kochenderfer, "Reluplex: An efficient SMT solver for verifying deep neural networks," in *Proc. CAV*, in Lecture Notes in Computer Science, vol. 10426. Cham, Switzerland: Springer, 2017, pp. 97–117.
- [85] J. Kauffmann, M. Esders, G. Montavon, W. Samek, and K.-R. Müller, "From clustering to cluster explanations via neural networks," 2019, *arXiv:1906.07633*. [Online]. Available: <http://arxiv.org/abs/1906.07633>
- [86] J. Kauffmann, K.-R. Müller, and G. Montavon, "Towards explaining anomalies: A deep Taylor decomposition of one-class models," *Pattern Recognit.*, vol. 101, May 2020, Art. no. 107198.
- [87] D. R. Kelley, Y. A. Reshef, M. Bileschi, D. Belanger, C. Y. McLean, and J. Snoek, "Sequential regulatory activity prediction across chromosomes with convolutional neural networks," *Genome Res.*, vol. 28, no. 5, pp. 739–750, May 2018.
- [88] J. Khan et al., "Classification and diagnostic prediction of cancers using gene expression profiling and artificial neural networks," *Nature Med.*, vol. 7, no. 6, pp. 673–679, Jun. 2001.
- [89] B. Kim, M. Wattenberg, J. Gilmer, C. J. Cai, J. Wexler, F. B. Viégas, and R. Sayres, "Interpretability beyond feature attribution: Quantitative testing with concept activation vectors (TCAV)," in *Proc. 35th Int. Conf. Mach. Learn.*, vol. 80, 2018, pp. 2673–2682.
- [90] P.-J. Kindermans et al., "Learning how to explain neural networks: PatternNet and PatternAttribution," in *Proc. 6th Int. Conf. Learn. Represent.*, 2018.
- [91] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. 5th Int. Conf. Learn. Represent.*, 2017.
- [92] C. Kittel, *Introduction to Solid State Physics*, vol. 8. New York, NY, USA: Wiley, 2004.
- [93] F. Klauschen et al., "Scoring of tumor-infiltrating lymphocytes: From visual estimation to machine learning," *Seminars Cancer Biol.*, vol. 52, pp. 151–157, Oct. 2018.
- [94] M. Kohlbrenner, A. Bauer, S. Nakajima, A. Binder, W. Samek, and S. Lapuschkin, "Towards best practice in explaining neural network decisions with LRP" in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2020, pp. 1–7.
- [95] N. Kokhlikyan et al. (2019). *Pytorch Captum*. [Online]. Available: <https://github.com/pytorch/captum>
- [96] K. Kourou, T. P. Exarchos, K. P. Exarchos, M. V. Karamouzis, and D. I. Fotiadis, "Machine learning applications in cancer prognosis and prediction," *Comput. Struct. Biotechnol. J.*, vol. 13, pp. 8–17, Jan. 2015.
- [97] F. Kratzert, M. Herrnegger, D. Klotz, S. Hochreiter, and G. Klambauer, "NeuralHydrology—Interpreting LSTMs in hydrology," in *Explainable AI: Interpreting, Explaining and Visualizing Deep Learning* (Lecture Notes in Computer Science), vol. 11700. Cham, Switzerland: Springer, 2019, pp. 347–362.
- [98] O. Z. Kraus, L. J. Ba, and B. J. Frey, "Classifying and segmenting microscopy images with deep multiple instance learning," *Bioinformatics*, vol. 32, no. 12, pp. 52–59, 2016.
- [99] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," in *Proc. NIPS*, 2012, pp. 1106–1114.
- [100] W. Landecker, M. D. Thomure, L. M. A. Bettencourt, M. Mitchell, G. T. Kenyon, and S. P. Brumby, "Interpreting individual classifications of hierarchical networks," in *Proc. IEEE Symp. Comput. Intell. Data Mining (CIDM)*, Apr. 2013, pp. 32–38.
- [101] S. Lapuschkin, A. Binder, G. Montavon, K.-R. Müller, and W. Samek, "Analyzing classifiers: Fisher vectors and deep neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 2912–2920.
- [102] S. Lapuschkin, A. Binder, G. Montavon, K.-R. Müller, and W. Samek, "The layer-wise relevance propagation toolbox for artificial neural networks," *J. Mach. Learn. Res.*, vol. 17, no. 114, pp. 1–5, 2016.
- [103] W. Samek, A. Binder, S. Lapuschkin, and K.-R. Müller, "Understanding and comparing deep neural networks for age and gender classification," in *Proc. IEEE Int. Conf. Comput. Vis. Workshops (ICCVW)*, Oct. 2017, pp. 38–1629.
- [104] S. Lapuschkin, S. Wäldchen, A. Binder, G. Montavon, W. Samek, and K.-R. Müller, "Unmasking clever Hans predictors and assessing what machines really learn," *Nature Commun.*, vol. 10, no. 1, p. 1096, 2019.
- [105] H. Larochelle and G. E. Hinton, "Learning to combine foveal glimpses with a third-order Boltzmann machine," in *Proc. Adv. Neural Inf. Process. Syst.* 23, 2010, pp. 1243–1251.
- [106] S. M. Lauritsen et al., "Explainable artificial intelligence model to predict acute critical illness from electronic health records," *Nature Commun.*, vol. 11, no. 1, Dec. 2020.
- [107] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, p. 436, 2015.
- [108] Y. A. LeCun, L. Bottou, G. B. Orr, and K.-R. Müller, "Efficient backprop," in *Neural Networks: Tricks of the Trade*. Berlin, Germany: Springer, 2012, pp. 9–48.
- [109] Y. Lee, C. D. Kummerow, and I. Ebert-Uphoff, "Applying machine learning methods to detect convection using GOES-16 ABI data," *Atmos. Meas. Techn. Discuss.*, vol. 2020, pp. 1–28, Nov. 2020.
- [110] K. Leino, S. Sen, A. Datta, M. Fredrikson, and L. Li, "Influence-directed explanations for deep convolutional networks," in *Proc. IEEE Int. Test Conf. (ITC)*, Oct. 2018, pp. 1–8.
- [111] M. Lin, Q. Chen, and S. Yan, "Network in network," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2014.
- [112] Z. C. Lipton, "The myths of model interpretability," *ACM Queue*, vol. 16, no. 3, p. 30, 2018.
- [113] H. Liu, Q. Yin, and W. Y. Wang, "Towards explainable NLP: A generative explanation framework for text classification," in *Proc. 57th Annu. Meeting Assoc. Comput. Linguistics*, 2019, pp. 5570–5581.
- [114] Y. Liu, K. Barr, and J. Reinitz, "Fully interpretable deep learning model of transcriptional control," *Bioinformatics*, vol. 36, no. 1, pp. i499–i507, Jul. 2020.
- [115] S. M. Lundberg et al., "From local explanations to global understanding with explainable ai for trees," *Nature Mach. Intell.*, vol. 2, no. 1, pp. 2522–5839, 2020.
- [116] S. M. Lundberg and S. Lee, "A unified approach to interpreting model predictions," in *Proc. Adv. Neural Inf. Process. Syst.* 30, 2017, pp. 4765–4774.
- [117] S. Ma, X. Song, and J. Huang, "Supervised group lasso with applications to microarray data analysis," *BMC Bioinformatics*, vol. 8, 2007.
- [118] J. Macdonald, S. Wäldchen, S. Hauch, and G. Kutyniok, "A rate-distortion framework for explaining neural network decisions," 2019, *arXiv:1905.11092*. [Online]. Available: <http://arxiv.org/abs/1905.11092>
- [119] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proc.*

- 5th Berkeley Symp. Math. Statist. Probab., vol. 1. Berkeley, CA, USA: Univ. of California Press, 1967, pp. 281–297.
- [120] Y. Matsui and T. Matsui, “NP-completeness for calculating power indices of weighted majority games,” *Theor. Comput. Sci.*, vol. 263, nos. 1–2, pp. 305–310, Jul. 2001.
- [121] A. McGovern et al., “Making the black box more transparent: Understanding the physical implications of machine learning,” *Bull. Amer. Meteorological Soc.*, vol. 100, no. 11, pp. 2175–2199, Nov. 2019.
- [122] R. Memisevic, “Learning to relate images,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 8, pp. 1829–1846, Aug. 2013.
- [123] T. Miller, “Explanation in artificial intelligence: Insights from the social sciences,” *Artif. Intell.*, vol. 267, pp. 1–38, Feb. 2019.
- [124] V. Mnih et al., “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [125] G. Montavon, “Gradient-based vs. propagation-based explanations: An axiomatic comparison,” in *Explainable AI: Interpreting, Explaining and Visualizing Deep Learning* (Lecture Notes in Computer Science), vol. 11700. Cham, Switzerland: Springer, 2019, pp. 253–265.
- [126] G. Montavon, A. Binder, S. Lapuschkin, W. Samek, and K.-R. Müller, “Layer-wise relevance propagation: An overview,” in *Explainable AI: Interpreting, Explaining and Visualizing Deep Learning* (Lecture Notes in Computer Science), vol. 11700. Cham, Switzerland: Springer, 2019, pp. 193–209.
- [127] G. Montavon, S. Lapuschkin, A. Binder, W. Samek, and K.-R. Müller, “Explaining nonlinear classification decisions with deep Taylor decomposition,” *Pattern Recognit.*, vol. 65, pp. 211–222, May 2017.
- [128] G. Montavon, W. Samek, and K.-R. Müller, “Methods for interpreting and understanding deep neural networks,” *Digit. Signal Process.*, vol. 73, pp. 1–15, Feb. 2018.
- [129] G. F. Montúfar, R. Pascanu, K. Cho, and Y. Bengio, “On the number of linear regions of deep neural networks,” in *Proc. Adv. Neural Inf. Process. Syst.* 27, 2014, pp. 2924–2932.
- [130] N. J. S. Mørch et al., “Visualization of neural networks using saliency maps,” in *Proc. Int. Conf. Neural Netw. (ICNN95)*, 1995, pp. 2085–2090.
- [131] A. Mordvintsev, C. Olah, and M. Tyka, “Inceptionism: Going deeper into neural networks,” *Google Res. Blog*, 2015.
- [132] K.-R. Müller, S. Mika, G. Rätsch, K. Tsuda, and B. Schölkopf, “An introduction to kernel-based learning algorithms,” *IEEE Trans. Neural Netw.*, vol. 12, no. 2, pp. 181–201, Mar. 2001.
- [133] M. Narayanan, E. Chen, J. He, B. Kim, S. Gershman, and F. Doshi-Velez, “How do humans understand explanations from machine learning systems? An evaluation of the human-interpretability of explanation,” 2018, *arXiv:1802.00682*. [Online]. Available: <http://arxiv.org/abs/1802.00682>
- [134] A. M. Nguyen, A. Dosovitskiy, J. Yosinski, T. Brox, and J. Clune, “Synthesizing the preferred inputs for neurons in neural networks via deep generator networks,” in *Proc. Adv. Neural Inf. Process. Syst.* 29, 2016, pp. 3387–3395.
- [135] A. M. Nguyen, J. Yosinski, and J. Clune, “Deep neural networks are easily fooled: High confidence predictions for unrecognizable images,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2015, pp. 427–436.
- [136] A. Nguyen, J. Yosinski, and J. Clune, “Multifaceted feature visualization: Uncovering the different types of features learned by each neuron in deep neural networks,” 2016, *arXiv:1602.03616*. [Online]. Available: <http://arxiv.org/abs/1602.03616>
- [137] R. Okuta, Y. Unno, D. Nishino, S. Hido, and C. Loomis, “CuPy: A numpy-compatible library for NVIDIA GPU calculations,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, p. 151.
- [138] T. E. Oliphant, *A Guide to NumPy*, vol. 1. Austin, TX, USA: Trelgol Publishing, 2006.
- [139] G. Papadopoulos, P. J. Edwards, and A. F. Murray, “Confidence estimation methods for neural networks: A practical comparison,” *IEEE Trans. Neural Netw.*, vol. 12, no. 6, pp. 1278–1287, Nov. 2001.
- [140] Y. Park, B. Kwon, J. Heo, X. Hu, Y. Liu, and T. Moon, “Estimating PM2.5 concentration of the conterminous united states via interpretable convolutional neural networks,” *Environ. Pollut.*, vol. 256, Jan. 2020, Art. no. 113395.
- [141] E. Parzen, “On estimation of a probability density function and mode,” *Ann. Math. Statist.*, vol. 33, no. 3, pp. 1065–1076, Sep. 1962.
- [142] V. Petsiuk, A. Das, and K. Saenko, “RISE: Randomized input sampling for explanation of black-box models,” in *Proc. Brit. Mach. Vis. Conf.*, 2018, p. 151.
- [143] B. Poulin et al., “Visual explanation of evidence with additive classifiers,” in *Proc. 21st Nat. Conf. Artif. Intell. 18th Innov. Appl. Artif. Intell. Conf.*, 2006, pp. 1822–1829.
- [144] K. Preuer, G. Klambauer, F. Rippmann, S. Hochreiter, and T. Unterthiner, “Interpretable deep learning in drug discovery,” in *Explainable AI: Interpreting, Explaining and Visualizing Deep Learning* (Lecture Notes in Computer Science), vol. 11700. Cham, Switzerland: Springer, 2019, pp. 331–345.
- [145] G. Quellec, K. Charrière, Y. Boudi, B. Cochener, and M. Lamard, “Deep image mining for diabetic retinopathy screening,” *Med. Image Anal.*, vol. 39, pp. 178–193, Jul. 2017.
- [146] S. Raghunath et al., “Prediction of mortality from 12-lead electrocardiogram voltage data using a deep neural network,” *Nature Med.*, vol. 26, no. 6, pp. 886–891, Jun. 2020.
- [147] M. T. Ribeiro, S. Singh, and C. Guestrin, “Why should I trust you?: Explaining the predictions of any classifier,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2016, pp. 1135–1144.
- [148] M. T. Ribeiro, S. Singh, and C. Guestrin, “Anchors: High-precision model-agnostic explanations,” in *Proc. AAAI Conf. Artif. Intell.*, vol. 18, 2018, pp. 1527–1535.
- [149] L. Rieger, P. Chormai, G. Montavon, L. Hansen, and K.-R. Müller, “Structuring neural networks for more explainable predictions,” in *Explainable and Interpretable Models in Computer Vision and Machine Learning*. Cham, Switzerland: Springer, 2019, pp. 115–131.
- [150] R. Roscher, B. Bohn, M. F. Duarte, and J. Garcke, “Explainable machine learning for scientific insights and discoveries,” *IEEE Access*, vol. 8, pp. 42200–42216, 2020.
- [151] F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychol. Rev.*, vol. 65, no. 6, pp. 386–408, 1958.
- [152] A. S. Ross, M. C. Hughes, and F. Doshi-Velez, “Right for the right reasons: Training differentiable models by constraining their explanations,” in *Proc. 26th Int. Joint Conf. Artif. Intell.*, 2017, pp. 2662–2670.
- [153] R. Rothe, R. Timofte, and L. Van Gool, “DEX: Deep expectation of apparent age from a single image,” in *Proc. IEEE Int. Conf. Comput. Vis. Workshops*, Dec. 2015, pp. 10–15.
- [154] C. Rudin, “Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead,” *Nature Mach. Intell.*, vol. 1, no. 5, pp. 206–215, May 2019.
- [155] L. Ruff et al., “A unifying review of deep and shallow anomaly detection,” *Proc. IEEE*, 2021, doi: [10.1109/JPROC.2021.3052449](https://doi.org/10.1109/JPROC.2021.3052449).
- [156] L. Ruff et al., “Deep one-class classification,” in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4393–4402.
- [157] O. Russakovsky et al., “ImageNet large scale visual recognition challenge,” *Int. J. Comput. Vis.*, vol. 115, no. 3, pp. 211–252, Dec. 2015.
- [158] W. Samek, A. Binder, G. Montavon, S. Lapuschkin, and K.-R. Müller, “Evaluating the visualization of what a deep neural network has learned,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 28, no. 11, pp. 2660–2673, Nov. 2017.
- [159] W. Samek, G. Montavon, A. Vedaldi, L. K. Hansen, and K.-R. Müller, Eds., *Explainable AI: Interpreting, Explaining and Visualizing Deep Learning* (Lecture Notes in Computer Science), vol. 11700. Cham, Switzerland: Springer, 2019.
- [160] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, “The graph neural network model,” *IEEE Trans. Neural Netw.*, vol. 20, no. 1, pp. 61–80, Jan. 2009.
- [161] J. Schmidhuber, “Deep learning in neural networks: An overview,” *Neural Netw.*, vol. 61, pp. 85–117, Jan. 2015.
- [162] T. Schnake et al., “Higher-order explanations of graph neural networks via relevant walks,” 2020, *arXiv:2006.03589*. [Online]. Available: <http://arxiv.org/abs/2006.03589>
- [163] B. Schölkopf and A. J. Smola, *Learning with Kernels: Support Vector Machines, Regularization, Optimization, and Beyond* (Adaptive Computation and Machine Learning Series). Cambridge, MA, USA: MIT Press, 2002.
- [164] B. Schölkopf, A. Smola, and K.-R. Müller, “Nonlinear component analysis as a kernel eigenvalue problem,” *Neural Comput.*, vol. 10, no. 5, pp. 1299–1319, Jul. 1998.
- [165] K. T. Schütt, F. Arbabzadah, S. Chmiela, K. R. Müller, and A. Tkatchenko, “Quantum-chemical insights from deep tensor neural networks,” *Nature Commun.*, vol. 8, no. 1, p. 13890, Apr. 2017.
- [166] K. T. Schütt, M. Gastegger, A. Tkatchenko, and K.-R. Müller, “Quantum-chemical insights from interpretable atomistic neural networks,” in *Explainable AI: Interpreting, Explaining and Visualizing Deep Learning* (Lecture Notes in Computer Science), vol. 11700. Cham, Switzerland: Springer, 2019, pp. 311–330.
- [167] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-CAM: Visual explanations from deep networks via gradient-based localization,” in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Oct. 2017, pp. 618–626.
- [168] C. Shan, “Learning local features for age estimation on real-life faces,” in *Proc. 1st ACM Int. workshop Multimodal Pervasive Video Anal. (MPVA)*, 2010, pp. 23–28.
- [169] L. S. Shapley, “17. A value for n-person games,” in *Contributions to Theory Games (AM-28)*, vol. 2. Princeton, NJ, USA: Princeton Univ. Press, 1953.
- [170] A. Shrikumar, P. Greenside, and A. Kundaje, “Learning important features through propagating activation differences,” in *Proc. 34th Int. Conf. Mach. Learn. (ICML)*, 2017, pp. 3145–3153.
- [171] A. Shrikumar, P. Greenside, A. Shcherbina, and A. Kundaje, “Not just a black box: Learning important features through propagating activation differences,” 2016, *arXiv:1605.01713*. [Online]. Available: <https://arxiv.org/abs/1605.01713>
- [172] A. Shrikumar, J. Su, and A. Kundaje, “Computationally efficient measures of internal neuron importance,” 2018, *arXiv:1807.09946*. [Online]. Available: <http://arxiv.org/abs/1807.09946>
- [173] M. Simon, E. Rodner, T. Darrell, and J. Denzler, “The whole is more than its parts? From explicit to implicit pose normalization,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 3, pp. 749–763, Mar. 2020.
- [174] K. Simonyan, A. Vedaldi, and A. Zisserman, “Deep inside convolutional networks: Visualising image classification models and saliency maps,” in *Proc. Int. Conf. Learn. Represent. Workshops*, 2014.
- [175] K. Simonyan and A. Zisserman, “Very deep

- convolutional networks for large-scale image recognition," in *Proc. 3rd Int. Conf. Learn. Represent.*, 2015.
- [176] D. Smilkov, N. Thorat, B. Kim, F. Viégas, and M. Wattenberg, "SmoothGrad: Removing noise by adding noise," 2017, [arXiv:1706.03825](https://arxiv.org/abs/1706.03825). [Online]. Available: <http://arxiv.org/abs/1706.03825>
- [177] C. Sonesson, S. Gerster, and M. Delorenzi, "Batch effect confounding leads to strong bias in performance estimates obtained by cross-validation," *PLoS ONE*, vol. 9, no. 6, Jun. 2014, Art. no. e100335.
- [178] J. T. Springenberg, A. Dosovitskiy, T. Brox, and M. A. Riedmiller, "Striving for simplicity: The all convolutional net," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.
- [179] E. Strumbelj and I. Kononenko, "An efficient explanation of individual classifications using game theory," *J. Mach. Learn. Res.*, vol. 11, pp. 1–18, Jan. 2010.
- [180] I. Sturm, S. Lapuschkin, W. Samek, and K.-R. Müller, "Interpretable deep neural networks for single-trial EEG classification," *J. Neurosci. Methods*, vol. 274, pp. 141–145, Dec. 2016.
- [181] P. Sturmels, S. Lundberg, and S.-I. Lee, "Visualizing the impact of feature attribution baselines," *Distill*, vol. 5, no. 1, Jan. 2020.
- [182] D. Su, H. Zhang, H. Chen, J. Yi, P. Chen, and Y. Gao, "Is robustness the cost of accuracy?—A comprehensive study on the robustness of 18 deep image classification models," in *Proc. 15th Eur. Conf. Comput. Vis.*, in Lecture Notes in Computer Science, vol. 11216. Cham, Switzerland: Springer, 2018, pp. 644–661.
- [183] M. Sundararajan, A. Taly, and Q. Yan, "Axiomatic attribution for deep networks," in *Proc. 34th Int. Conf. Mach. Learn.*, 2017, pp. 3319–3328.
- [184] W. R. Swartout and J. D. Moore, "Explanation in second generation expert systems," in *Proc. 2nd Gener. Expert Syst.* Berlin, Germany: Springer, 1993, pp. 543–585.
- [185] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. J. Goodfellow, and R. Fergus, "Intriguing properties of neural networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2014.
- [186] A. W. Thomas, H. R. Heekeren, K.-R. Müller, and W. Samek, "Analyzing neuroimaging data through recurrent deep learning models," *Frontiers Neurosci.*, vol. 13, p. 1321, Dec. 2019.
- [187] H. Trauttmüller and A. Eriksson, "The frequency range of the voice fundamental in the speech of male and female adults," Institutionen lingvistik, Stockholms Univ., Stockholm, Sweden, Tech. Rep., 1995.
- [188] M. Tsang, D. Cheng, and Y. Liu, "Detecting statistical interactions from neural network weights," in *Proc. 6th Int. Conf. Learn. Represent.*, 2018.
- [189] B. Ustun, A. Spangher, and Y. Liu, "Actionable recourse in linear classification," in *Proc. Conf. Fairness, Accountability, Transparency*, 2019, pp. 10–19.
- [190] V. Vapnik, *The Nature of Statistical Learning Theory*. New York, NY, USA: Springer, 1995.
- [191] M. M.-C. Vidovic, N. Görnitz, K.-R. Müller, G. Rätsch, and M. Kloft, "Opening the black box: Revealing interpretable sequence motifs in kernel-based learning algorithms," in *Proc. Joint Eur. Conf. Mach. Learn. Knowl. Discovery Databases*. Springer, 2015, pp. 137–153.
- [192] C. Von Der Malsburg, "Binding in models of perception and brain function," *Current Opinion Neurobiol.*, vol. 5, no. 4, pp. 520–526, Aug. 1995.
- [193] A. Warnecke, D. Arp, C. Wressnegger, and K. Rieck, "Evaluating explanation methods for deep learning in security," 2019, [arXiv:1906.02108](https://arxiv.org/abs/1906.02108). [Online]. Available: <http://arxiv.org/abs/1906.02108>
- [194] C. K. Williams and C. E. Rasmussen, *Gaussian Processes for Machine Learning*. Cambridge, MA, USA: MIT Press, 2006.
- [195] K. Xu et al., "Show, attend and tell: Neural image caption generation with visual attention," in *Proc. 32nd Int. Conf. Mach. Learn.*, vol. 37, 2015, pp. 2048–2057.
- [196] Y. Yang, V. Tresp, M. Wunderle, and P. A. Fasching, "Explaining therapy predictions with layer-wise relevance propagation in neural networks," in *Proc. IEEE Int. Conf. Healthcare Informat. (ICHI)*, Jun. 2018, pp. 152–162.
- [197] I.-C. Yeh, "Modeling of strength of high-performance concrete using artificial neural networks," *Cement Concrete Res.*, vol. 28, no. 12, pp. 1797–1808, Dec. 1998.
- [198] Z. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec, "GNNExplainer: Generating explanations for graph neural networks," in *Proc. Adv. Neural Inf. Process. Syst.* 32, 2019, pp. 9240–9251.
- [199] K. Young, G. Booth, B. Simpson, R. Dutton, and S. Shrapnel, "Deep neural network or dermatologist?" in *Interpretability of Machine Intelligence in Medical Image Computing and Multimodal Learning for Clinical Decision Support*. Cham, Switzerland: Springer, 2019, pp. 48–55.
- [200] T. Zahavy, N. Ben-Zrihem, and S. Mannor, "Graying the black box: Understanding DQNs," in *Proc. 33rd Int. Conf. Mach. Learn.*, vol. 48, 2016, pp. 1899–1908.
- [201] M. D. Zeiler and R. Fergus, "Visualizing and understanding convolutional networks," in *Proc. Eur. Conf. Comput. Vis.-ECCV*, 2014, pp. 818–833.
- [202] J. Zhang, S. A. Bargal, Z. Lin, J. Brandt, X. Shen, and S. Sclaroff, "Top-down neural attention by excitation backprop," *Int. J. Comput. Vis.*, vol. 126, no. 10, pp. 1084–1102, Oct. 2018.
- [203] Q. Zhang, X. Wang, R. Cao, Y. N. Wu, F. Shi, and S.-C. Zhu, "Extracting an explanatory graph to interpret a CNN," *IEEE Trans. Pattern Anal. Mach. Intell.*, early access, May 4, 2020, doi: [10.1109/TPAMI.2020.2992207](https://doi.org/10.1109/TPAMI.2020.2992207).
- [204] Z. Zhang et al., "Pathologist-level interpretable whole-slide cancer diagnosis with deep learning," *Nature Mach. Intell.*, vol. 1, no. 5, pp. 236–245, May 2019.
- [205] B. Zhou, D. Bau, A. Oliva, and A. Torralba, "Interpreting deep visual representations via network dissection," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, no. 9, pp. 2131–2145, Sep. 2019.
- [206] B. Zhou, A. Khosla, A. Lapedriza, A. Oliva, and A. Torralba, "Learning deep features for discriminative localization," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 2921–2929.
- [207] X.-P. Zhu, J.-M. Dai, C.-J. Bian, Y. Chen, S. Chen, and C. Hu, "Galaxy morphology classification with deep convolutional neural networks," *Astrophys. Space Sci.*, vol. 364, no. 4, p. 55, Apr. 2019.
- [208] L. M. Zintgraf, T. S. Cohen, T. Adel, and M. Welling, "Visualizing deep neural network decisions: Prediction difference analysis," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017.

## ABOUT THE AUTHORS

**Wojciech Samek** (Member, IEEE) studied computer science at Humboldt University of Berlin, Berlin, Germany, from 2004 to 2010. He received the Ph.D. degree with distinction from the Technical University of Berlin, Berlin, in 2014.

During his studies, he was awarded scholarships from the German Academic Scholarship Foundation and the DFG Research Training Group GRK 1589/1. He was a Visiting Researcher with the NASA Ames Research Center, Mountain View, CA, USA. In 2014, he founded the Machine Learning Group, Fraunhofer Heinrich Hertz Institute, Berlin, which he has directed until 2020. He is currently the Head of the Department of Artificial Intelligence and the Explainable AI Group, Fraunhofer Heinrich Hertz Institute. He is also an Associate Faculty with the Berlin Institute for the Foundations of Learning and Data (BIFOLD), Berlin, the ELLIS Unit Berlin, Berlin, and the DFG Graduate School BIOQIC, Berlin. His research interests include deep learning, explainable AI, neural network compression, and federated learning.

Dr. Samek is an Editorial Board Member of *Pattern Recognition*, *PLoS ONE*, and IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS and an Elected Member of the IEEE MLSP Technical Committee. He was a recipient of multiple best paper awards, including the 2020 Pattern Recognition Best Paper Award, and a



part of the MPEG-7 Part 17 standardization. He has been serving as an AC for Annual Conference of the North American Chapter of the Association for Computational Linguistics (NAACL) in 2021.

**Grégoire Montavon** received the master's degree in communication systems from the École Polytechnique Fédérale de Lausanne, Lausanne, Switzerland, in 2009, and the Ph.D. degree in machine learning from the Technische Universität Berlin, Berlin, Germany, in 2013.

He is currently a Senior Researcher with the Machine Learning Group, Technische Universität Berlin, and the Berlin Institute for the Foundations of Learning and Data (BIFOLD), Berlin. He is also a member of the ELLIS Unit Berlin, Berlin. His research interests include explainable machine learning, deep neural networks, and unsupervised learning.

Dr. Montavon is an Editorial Board Member of *Pattern Recognition*. He was a recipient of the 2020 Pattern Recognition Best Paper Award.



**Sebastian Lapuschkin** received the Ph.D. degree with distinction from the Berlin Institute of Technology, Berlin, Germany, in 2018, for his pioneering contributions to the field of eXplainable Artificial Intelligence and interpretable machine learning.

From 2007 to 2013, he studied computer science (B.Sc. and M.Sc.) at the Berlin Institute of Technology, with a focus on software engineering and machine learning. He is currently a tenured Researcher and the Head of the Explainable AI Group, Fraunhofer Heinrich Hertz Institute (HHI), Berlin. His research is focused on computer vision, (efficient) machine learning and data analysis, data and algorithm visualization, and the interpretation, holistic analysis, and rectification of machine learning system behavior.

Dr. Lapuschkin was a recipient of multiple awards, including the Hugo-Geiger-Prize for outstanding doctoral achievement and the 2020 Pattern Recognition Best Paper Award.



**Christopher J. Anders** received the B.Sc. and M.Sc. degrees in computer science from Technische Universität Berlin, Berlin, Germany, in 2016 and 2018, respectively, with a focus on machine learning and computer security. He is currently working toward the Ph.D. degree, as a Junior Researcher, at the Machine Learning Group, Technische Universität Berlin, and the Berlin Institute for the Foundations of Learning and Data (BIFOLD), Berlin.

His research interests include (adversarial) explainable machine learning and deep neural networks.



**Klaus-Robert Müller** (Member, IEEE) studied physics at the Technische Universität Karlsruhe, Karlsruhe, Germany, from 1984 to 1989. He received the Ph.D. degree in computer science from Technische Universität Karlsruhe in 1992.

After completing a postdoctoral position at GMD FIRST, Berlin, Germany, he was a Research Fellow with The University of Tokyo, Tokyo, Japan, from 1994 to 1995. In 1995, he founded the Intelligent Data Analysis Group, GMD-FIRST (later Fraunhofer FIRST), and directed it until 2008. From 1999 to 2006, he was a Professor with the University of Potsdam, Potsdam, Germany. He has been a Professor of computer science with Technische Universität Berlin, Berlin, since 2006; at the same time, he is also codirecting the Berlin Big Data Center, Berlin, and the Berlin Institute for the Foundations of Learning and Data (BIFOLD), Berlin. His research interests are intelligent data analysis and machine learning in the sciences (neuroscience (specifically brain-computer interfaces), physics, and chemistry).

Prof. Müller was an Elected Member of the German National Academy of Sciences-Leopoldina in 2012 and the Berlin Brandenburg Academy of Sciences in 2017 and an External Scientific Member of the Max Planck Society in 2017. In 2019 and 2020, he became a Highly Cited Researcher in the cross-disciplinary area. He was awarded the Olympus Prize for Pattern Recognition in 1999, the SEL Alcatel Communication Award in 2006, the Science Prize of Berlin by the Governing Mayor of Berlin in 2014, and the Vodafone Innovations Award in 2017.

